



Ostfalia
Hochschule für angewandte
Wissenschaften

Modellierung und UML

Zustandsdiagramme

Prof. Dr. Michaela Huhn und Prof. Dr. Sebastian Bindick

Gliederung

- **Grundlagen**

- Was ist ein Zustandsdiagramm?
- Anwendungsbereiche von Zustandsdiagrammen

- **UML-Notationselemente im Detail**

- Zustand
- Event und Transition
- Startzustand, Endzustand und Terminator
- Entscheidung und Kreuzung
- Zusammengesetzter Zustand
- Region
- Rahmen eines Zustandsautomaten

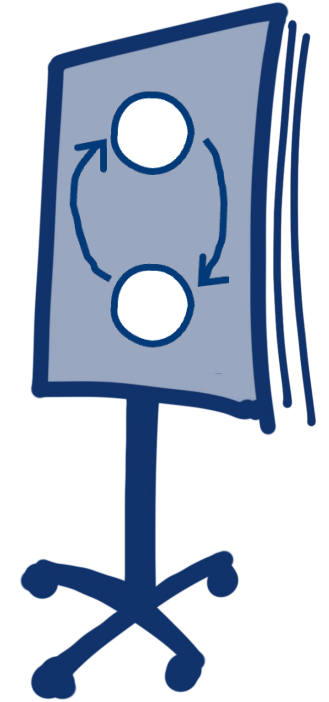
- **Zustandsdiagramme in Java**

ZUSTANDSDIAGRAMME IM ÜBERBLICK

Was ist ein Zustandsdiagramm?

„**Zustandsdiagramme** (engl. State Machine Diagrams) modellieren wie Aktivitätsdiagramme das **dynamische Verhalten eines Systems**. Im Gegensatz zu Aktivitätsdiagrammen, die ihren Fokus auf die Aktionen eines Systems legen, konzentrieren sich Zustandsdiagramme auf die **Reaktionen eines Systems**.“

- Kecher*



*Quelle: UML 2.5 - Das umfassende Handbuch, Kecher, Hoffmann-Elbern, Will, Rheinwerk, 2021

Wer hat's erfunden?

David Harel (*12 April 1950 in London),

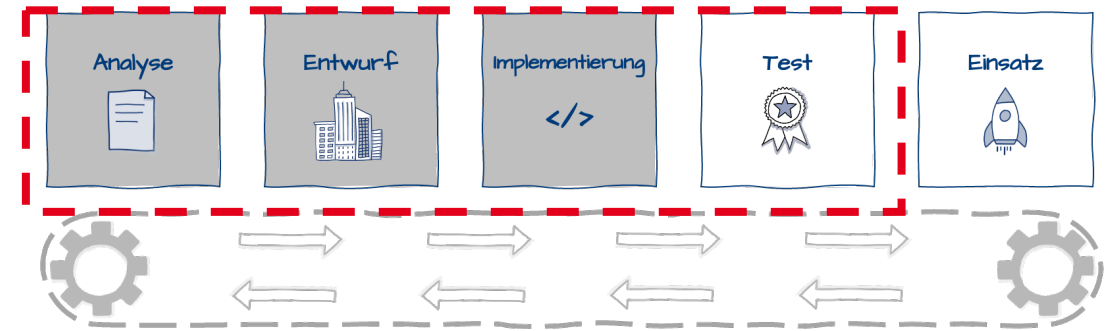
- israelischer Informatiker, Professor am Weizmann Institut
- Präsident der israelischen Akademie der Wissenschaften
- „Erfinder“ von Statecharts
 - Erste praxistaugliche Automaten-Modellierungsnotation
 - ACM Software System Award (2007)
 - IEEE Harlan D. Mills Award (2023)



Statecharts sind die Basisnotation für eine Vielzahl von Werkzeugen zur Modellierung, Simulation und Codegenerierung von Steuerungen.

Anwendungsbereiche von Zustandsdiagrammen

- Zustandsdiagramme werden zur Ergänzung von Klassendiagrammen eingesetzt
- Verwendung v. a. in Analyse- und Entwurfsphase
- Fokus: Lebenszyklus („Lebensweg“) von Objekten
- Jedes Objekt befindet sich jederzeit in einem Zustand
- Zustand wird durch Attributwerte bestimmt
- Objekte durchlaufen nur sinnvolle Zustände während ihrer Lebensdauer
- Zustandsdiagramme modellieren diese zulässigen Zustände und Übergänge
 - und können simuliert und (automatisiert) in Code überführt werden
 - oder zur Erzeugung von Testfällen benutzt werden

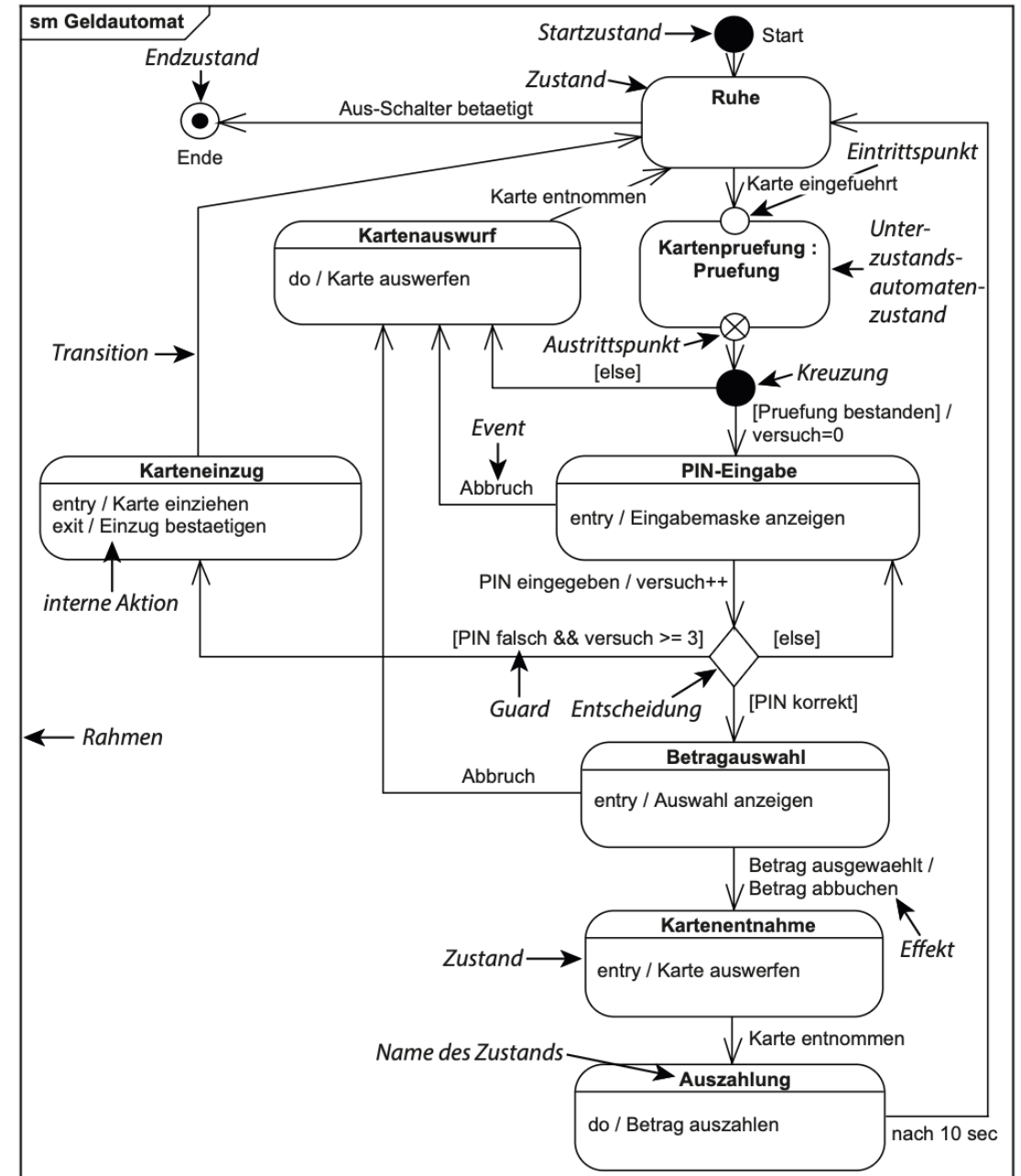


Zustandsdiagramme im Überblick

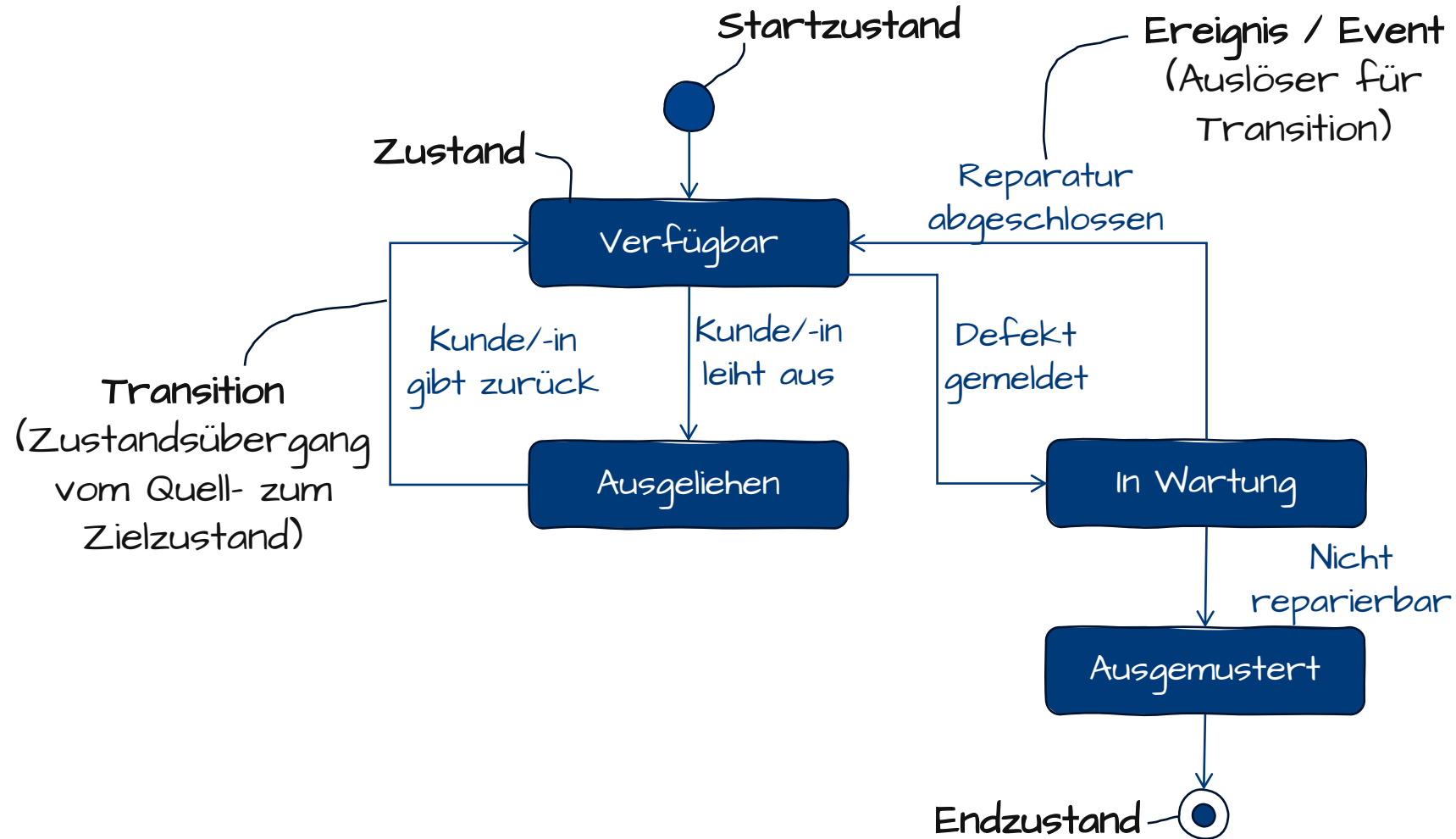


Mit einem **Zustandsdiagramm** werden die **Reaktionen** eines **Systems, Subsystems oder Objekts** und die damit einhergehenden **Zustandsänderungen** modelliert.

Während sich Aktivitätsdiagramme auf Abläufe, Workflows oder Prozesse konzentrieren, modellieren Zustandsdiagramme das vollständige Verhalten einer einzelnen Entität über ihren gesamten Lebenszyklus.



Zustandsdiagramme im Überblick

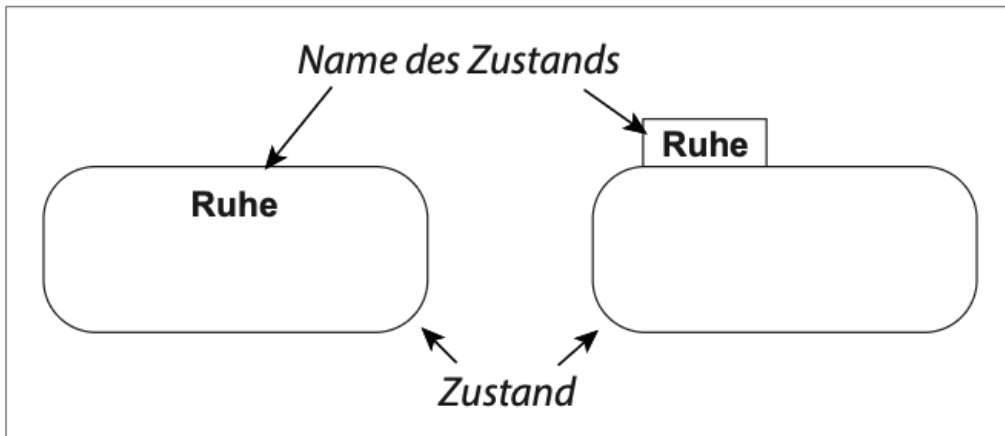


UML-NOTATIONSELEMENT: ZUSTAND

Zustand



„Ein **Zustand** (engl. Simple State) **modelliert eine Situation**, in der gewisse, **genau definierte Bedingungen** gelten.“ – Kecher*



Bildquelle Kecher*

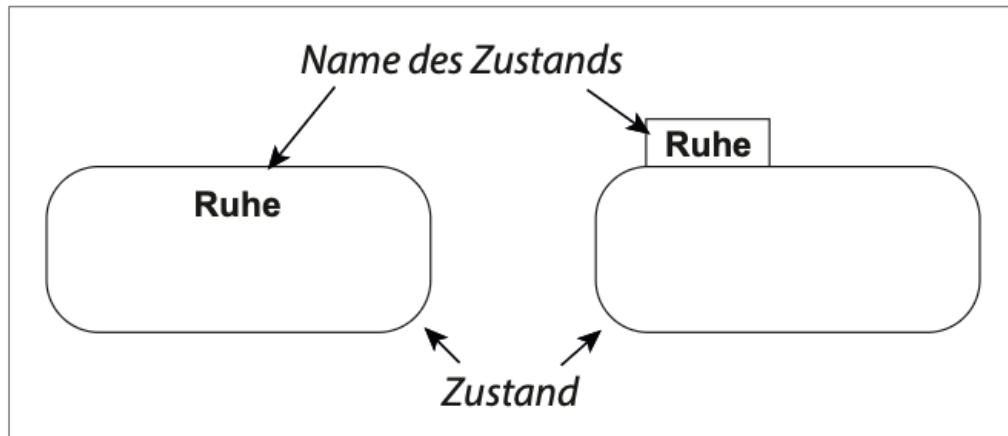
Ein Zustand beschreibt entweder eine statische Situation (z. B. Warten auf eine Eingabe) oder eine dynamische Aktivität (z. B. das Ausführen einer Aufgabe).

Der aktuell eingenommene Zustand eines Objekts wird als **aktiver Zustand** bezeichnet.

Zustand



„Ein **Zustand** (engl. Simple State) **modelliert eine Situation**, in der gewisse, **genau definierte Bedingungen** gelten.“ – Kecher*



Bildquelle Kecher*

Ein Zustand beschreibt eine Konfiguration* oder einen Status eines Systems oder Objekts, die in der Regel über einen gewissen Zeitraum bestehen. Zustände modellieren temporär stabile Situationen (z. B. „betriebsbereit und wartet auf eine Eingabe“) oder kontinuierliche Aktivitäten oder Prozesse (z. B. „drucken“ oder „aufheizen“).

Der aktuell eingenommene Zustand eines Objekts wird als **aktiver Zustand** bezeichnet.

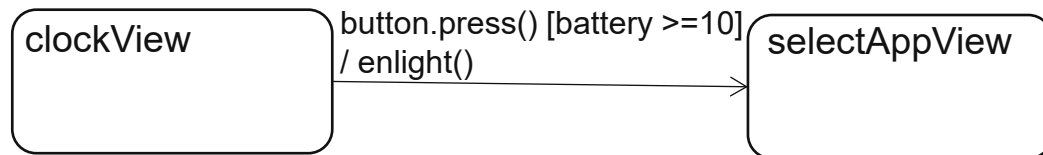
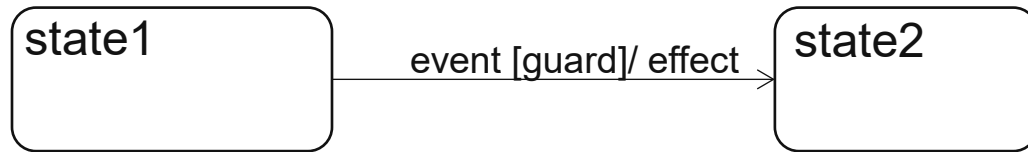
* Konfiguration: Eine spezifische Ausprägung von Systemparametern.

UML-NOTATIONSELEMENT: TRANSITION

Transition



„Eine **Transition** (engl. Transition) ist eine gerichtete Beziehung zwischen zwei Zuständen und stellt einen **Zustandsübergang vom Quell- zum Zielzustand** dar.“ – Kecher*

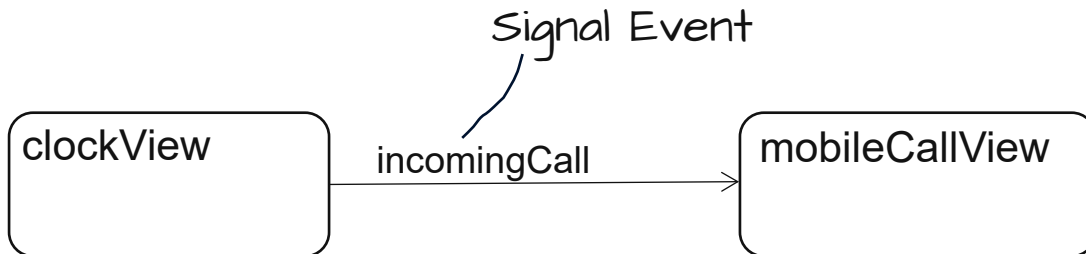
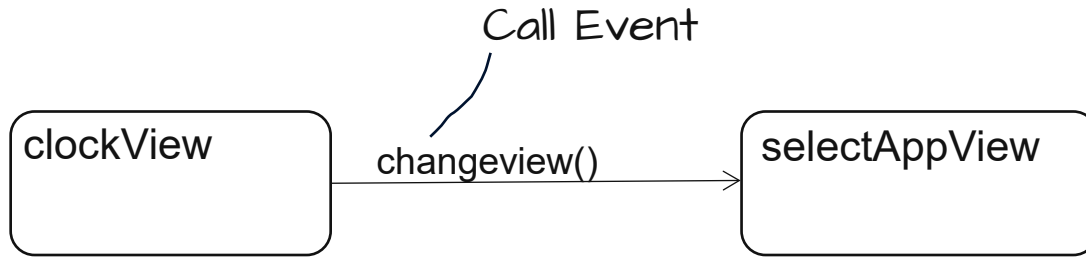


Beispiele inspiriert von D. Harel's Digital Watch**, angewendet auf eine Smart Watch

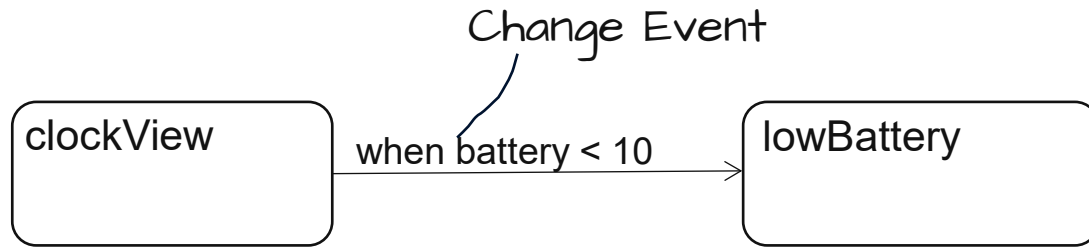
- Transitionen modellieren den Übergang zwischen Zuständen
- Sie werden optional beschriftet mit:
 - **event [guard] / effect**
- Ein Objekt reagiert auf das Auftreten eines Ereignisses
 - wenn im aktiven Zustand für dieses Ereignis eine Reaktion vorgesehen ist und der Guard erfüllt ist
 - andernfalls ... (→ später).
- Transitionen sind (idealisiert) zeitlos oder augenblicklich (engl. instantaneous).
 - **Wenn offensichtlich nicht: Modellierungsproblem!**

**Quelle: Harel, D. Statecharts: A Visual Formalism for Complex Systems. Science of Computer Programming 8 (1987) p. 231-274 Commun.

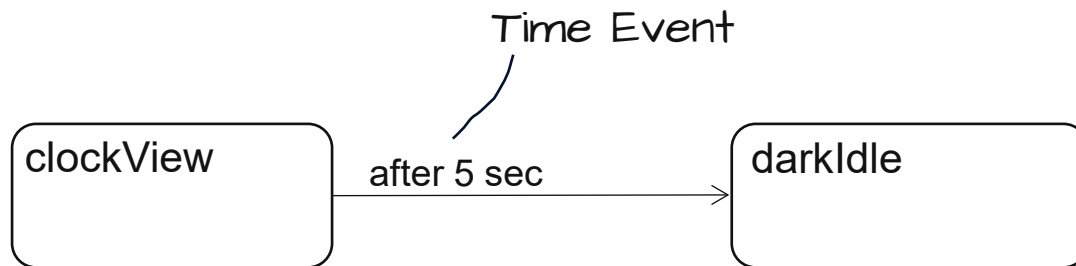
Ereignisse (Events) aktivieren eine Transition.



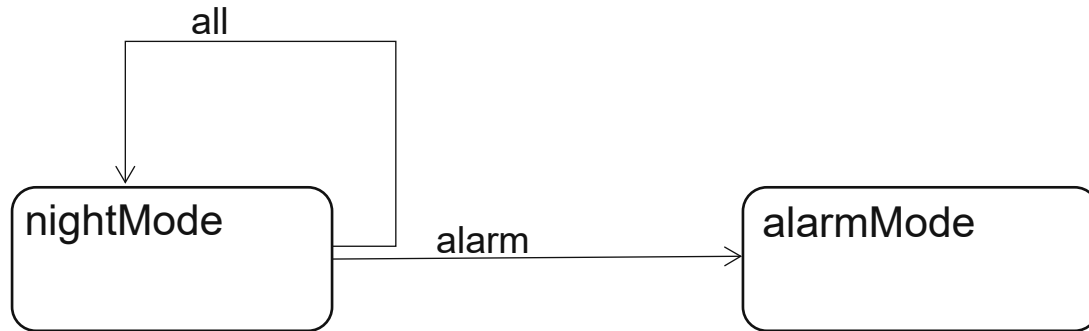
- **Call Event:** Anfrage zur Ausführung einer Operation
- Objekt führt die angeforderte Operation aus.
- I.d.R.: Zustandswechsel (Transition)
- Ausnahme: interne Transition (→später)
- **Signal Event:** Empfang eines asynchronen Signals
- Event „feuert“, sobald Signal eintrifft
- Objekt reagiert ggf. mit Zustandswechsel



- **Change Event:** durch Zustandsbedingung (boolescher Ausdruck) ausgelöst
- Notation: `when` (Bedingung)
- Event tritt ein, wenn Ausdruck von `false` → `true` wechselt



- **Time Event:** wird zu einem bestimmten Zeitpunkt oder nach einer Zeitspanne ausgelöst
- Reaktion erfolgt nach Ablauf der Zeit

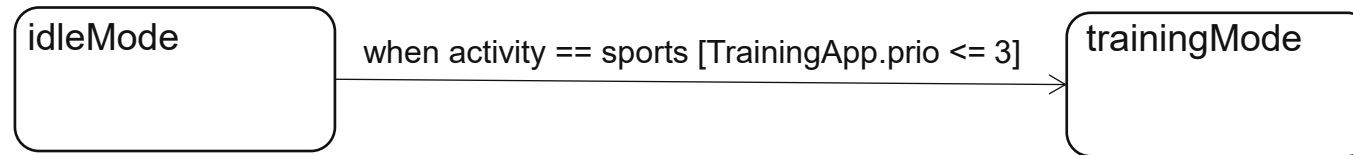


- **Any Receive Event (all)**: reagiert auf alle nicht explizit behandelten Events
- Wird genutzt als explizite Default-Reaktion für alle nicht genannten Events
- **Any-Receive-Transition wird nachgeordnet ausgewählt**, d.h. nur, wenn keine Transition mit explizitem Event aktiviert ist.

Guard



„Eine **Transition** wird nur **aktiviert**, wenn ihr **Guard** zu true ausgewertet wird.“

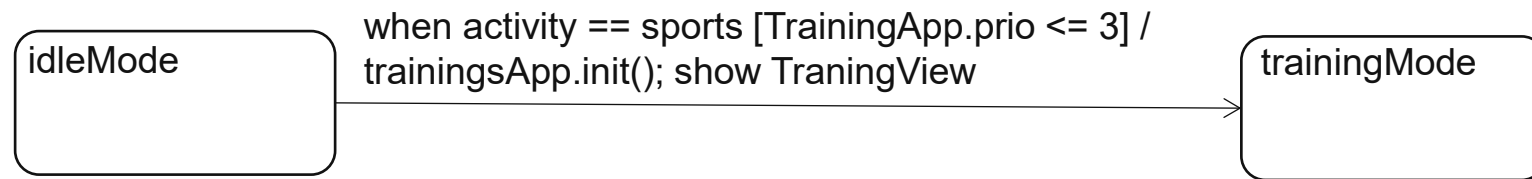


- Guard: Bedingung für die Ausführung einer Transition
- Notation: `[boolescher Ausdruck]`
- Transition wird nur aktiviert, wenn beim Verarbeiten des Ereignisses der Guard zu true ausgewertet wird
- Unterschied zu Change Event: Guard prüft, löst aber nicht aus
- Beispiel: `activity == sports`: change event tritt auf, aber `[TrainingApp.prio <= 3] == false` → Transition nicht aktiviert → kein Zustandswechsel

Effekt



„Ein **Effekt** definiert **Aktionen**, die bei einer **Transition ausgeführt** werden, und wird nach einem **Schrägstrich notiert**. “ – Kecher*

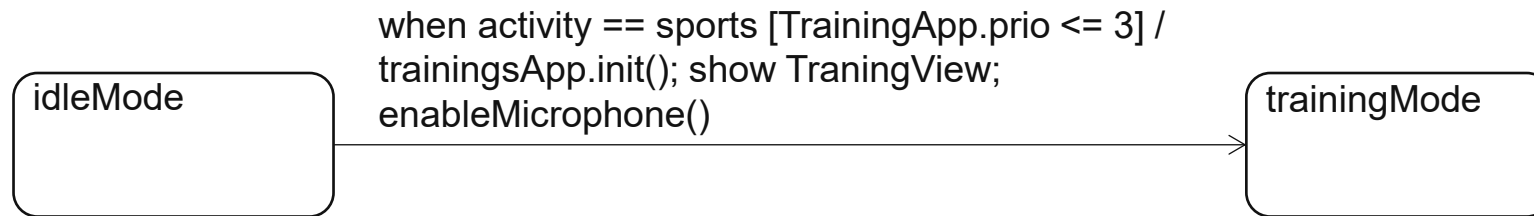


Sequenz von Aktionen

Effekt kann mehrere
Aktionen enthalten
(Aktionssequenz)

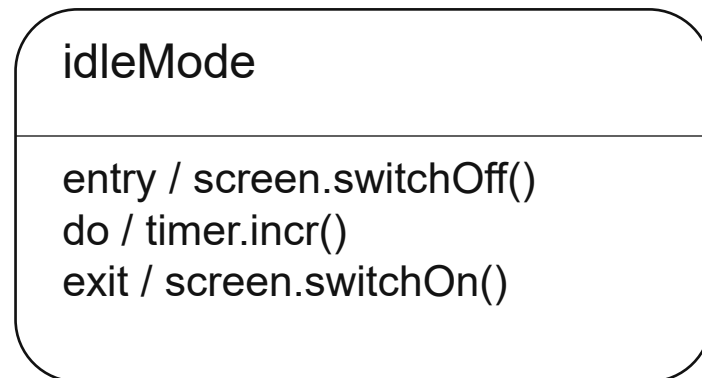


Grundsätzlich muss die Modellierung gewährleisten, dass Ereignisse vollständig verarbeitet werden, bevor das nächste Ereignis ansteht.



Spezielle interne Aktionen:

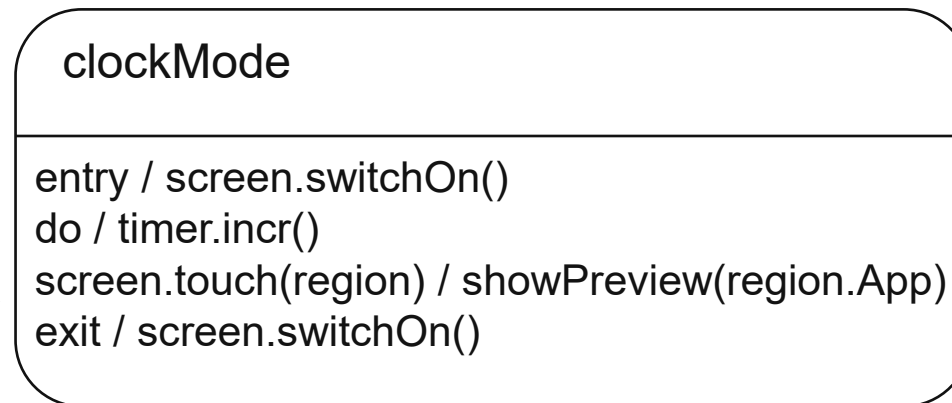
- **entry**: wird beim Betreten des Zustands ausgeführt
- **do**: läuft während des Zustands (bis es von sich aus endet oder der Zustand wechselt)
- **exit**: wird vor dem Verlassen des Zustands ausgeführt



Spezielle interne Aktionen:

- **Interne Transition:** Aktion wird ausgeführt, Zustand bleibt!
 - Sinnvoll, wenn es sich um seiteneffektfreie oder Hilfsfunktionen handelt, die am Status des Systems / Objekts nichts ändern

Interne
Transition



Mögliche Reaktionen auf das Auftreten eines Events:

Fall 1: Das Event steht als Trigger an einer internen oder ausgehenden Transition des aktiven Zustands

- **Transition wird aktiviert:** Wechsel in einen anderen Zustand
- **Interne Aktion:** Aktion wird ausgeführt, Zustand bleibt gleich
- *Offen: was passiert, wenn das Event als Trigger an verschiedenen Transitionen steht? → später*

Fall 2: Das Event steht **nicht als Trigger** an einer Transition des aktiven Zustands.

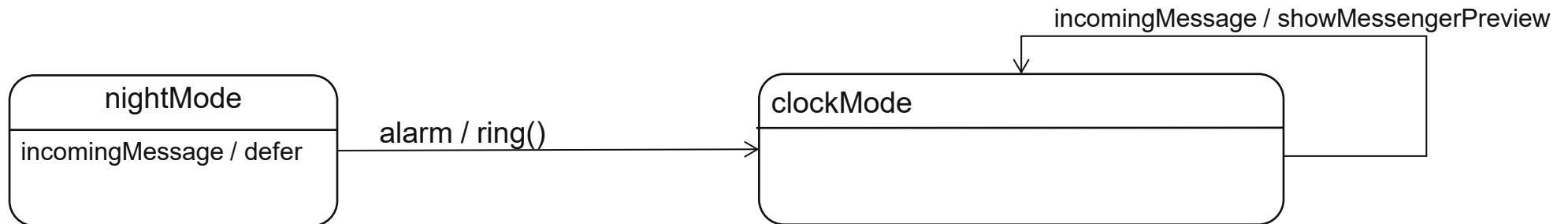
Die UML erlaubt unterschiedliche Interpretationen:

- **Ignorieren:** Event wird konsumiert, keine Reaktion
 - **Fehlermeldung** (z.B. als Anzeige) „Message not understood“, aber keine Zustandsänderung
 - **Spezifischer Fehlerzustand** wird eingenommen (und nur durch Rücksetznachricht verlassen).
 - **(Chaos:** Beliebige Reaktion erlaubt einen beliebigen Zustandsübergang und eine beliebige Aktion.)
-
- Die Varianten Ignorieren, Fehlermeldung, Fehlerzustand sind für eine **Implementierung** geeignet: Codegenerator!
 - Variante Chaos ist nur bei der Verwendung von Statecharts in einer (frühen) **Spezifikationsphase** geeignet.
 - Chaos erlaubt die robuste Implementierung durch spätere Entwurfsentscheidungen (Hinzufügen von Transitionen)
 - Chaos = Nicht-Wissen entspricht Unterspezifikation, die in späterem Entwurf aufgelöst wird

Ausnahme: Verzögern von Events

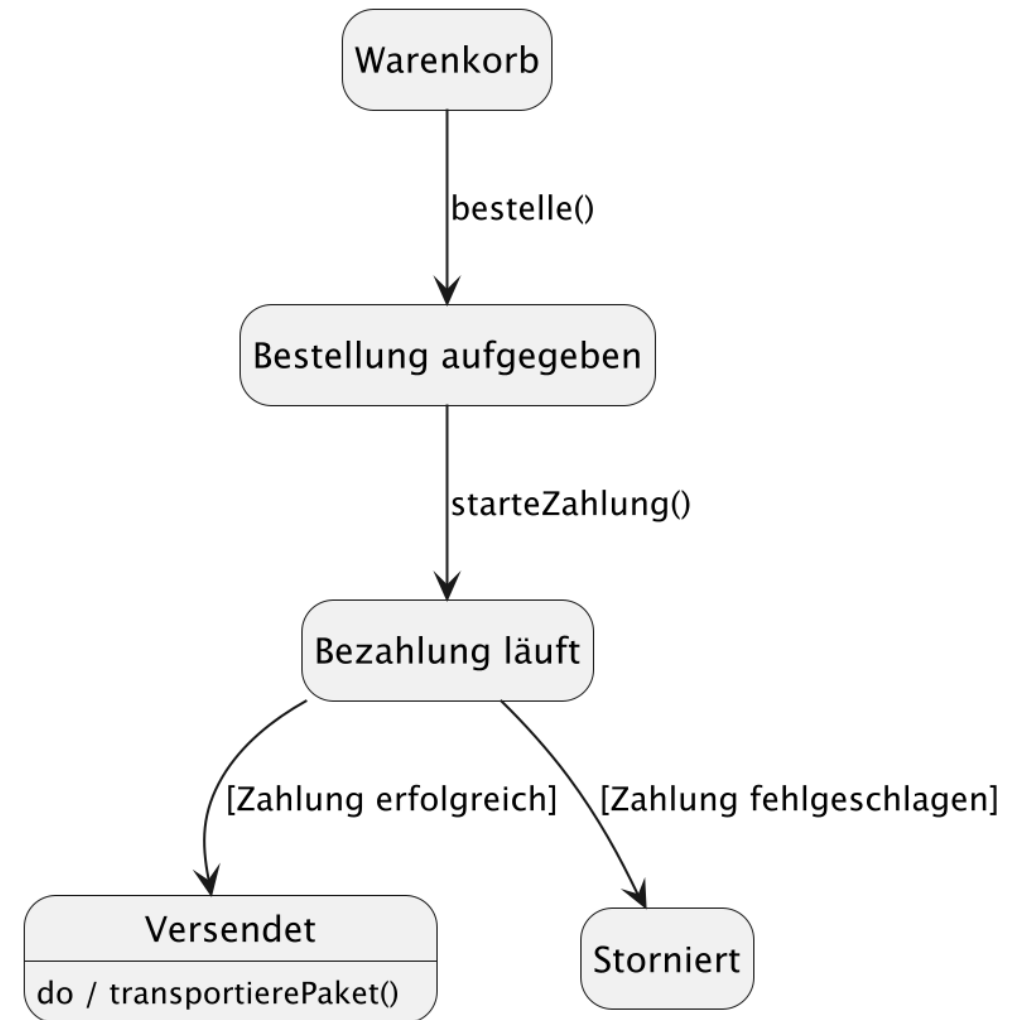


„Ein **bestimmtes Event kann verzögert** (engl. deferred) werden, falls im jeweiligen Zustand **nicht darauf reagiert** werden soll. Das **Event** wird **aufbewahrt** und bei **jedem Zustandswechsel** dem **neuen Zustand angeboten**, bis das Objekt einen Zustand erreicht, in dem das verzögerte Event eine Reaktion auslöst und das Event damit »verbraucht« wird. “ – Kecher*



Aufgabe 1: Welche Aussagen treffen zu?

- A** Die Transition von Warenkorb zu Bestellung aufgegeben wird durch ein Call Event ausgelöst.
- B** Die Bedingung [Zahlung erfolgreich] ist ein Guard.
- C** Im Zustand Versendet wird eine laufende Aktivität ausgeführt.
- D** Ein Wechsel von Versendet zurück zu Bezahlung läuft ist möglich.
- E** Die Transition von Bezahlung läuft zu Storniert wird durch ein Change Event ausgelöst.



Plantuml: Code Beispiel

@startuml

hide empty description

[*] -> Warenkorb

state "Bestellung aufgeben" as Bestellung

state "Bezahlung läuft" as Bezahlen

Warenkorb --> Bestellung: bestelle()

Bestellung --> Bezahlen: starteZahlung()

state Versendet : do / versende Paket()

state Storniert

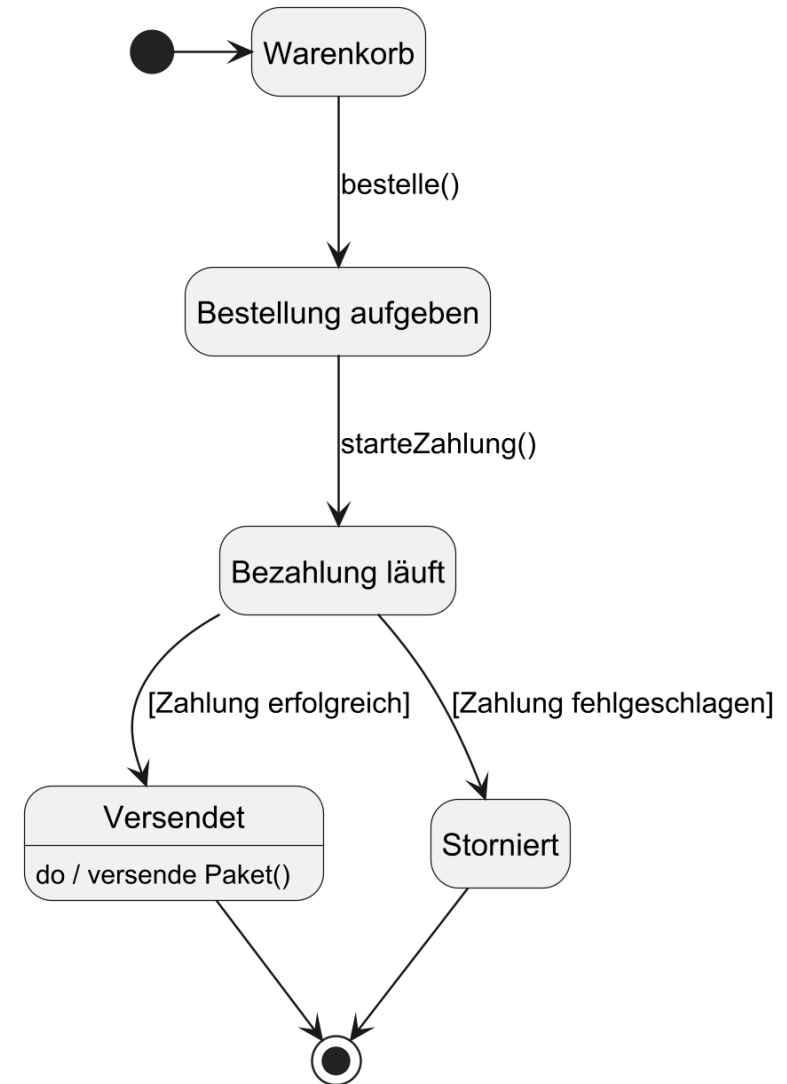
Bezahlen --> Versendet: [Zahlung erfolgreich]

Bezahlen --> Storniert: [Zahlung fehlgeschlagen]

Storniert --> [*]

Versendet --> [*]

@enduml





ÜBUNGSaufGABE 6.1

Lebenszyklus eines Kinotickets

Übungsaufgabe 6.1: Lebenszyklus eines Kinotickets

Modellieren Sie ein Zustandsdiagramm für den Lebenszyklus eines Kinotickets in einem Kinoticketsystem.

Berücksichtigen Sie dabei folgende Anforderungen:

- Ein Kinoticket ist zunächst im Zustand *Verfügbar*.
- Wählt ein/e Kunde/-in das Ticket aus, wird es *Reserviert*.
 - Ein reserviertes Ticket ist nach Ablauf einer Frist von 10 min wieder *Verfügbar* werden.
- Mit dem Call Event `starteBezahlung()` erreicht das Ticket den Zustand *Bezahlvorgang läuft*.
- Verläuft die Zahlung erfolgreich, wechselt das Ticket in den Zustand *Bezahlt*. Scheitert die Bezahlung, wird das Ticket wieder *Verfügbar*.
- Bis 2h vor Filmbeginn kann ein bezahltes Ticket storniert werden und ist dann wieder *Verfügbar*.
- Beim Einlass wird ein bezahltes Ticket beim Scannen *Entwertet*.
- 30 min nach Beginn des Films, wird das Ticket auch *Entwertet*.

Modellieren Sie die Zustände und Transitionen mit geeigneten Beschriftungen.

UML-NOTATIONSELEMENT: PSEUDO-ZUSTÄNDE: STARTZUSTAND, ENZZUSTAND UND TERMINATOR

Startzustand, Endzustand und Terminator



„Der **Startzustand** (engl. Initial) stellt den **Startpunkt des Zustandsautomaten** dar.

Die Ausführung einer **Region** oder **Ebene** von **Zuständen** ist beim Erreichen eines **Endzustands** (engl. Final State) beendet.

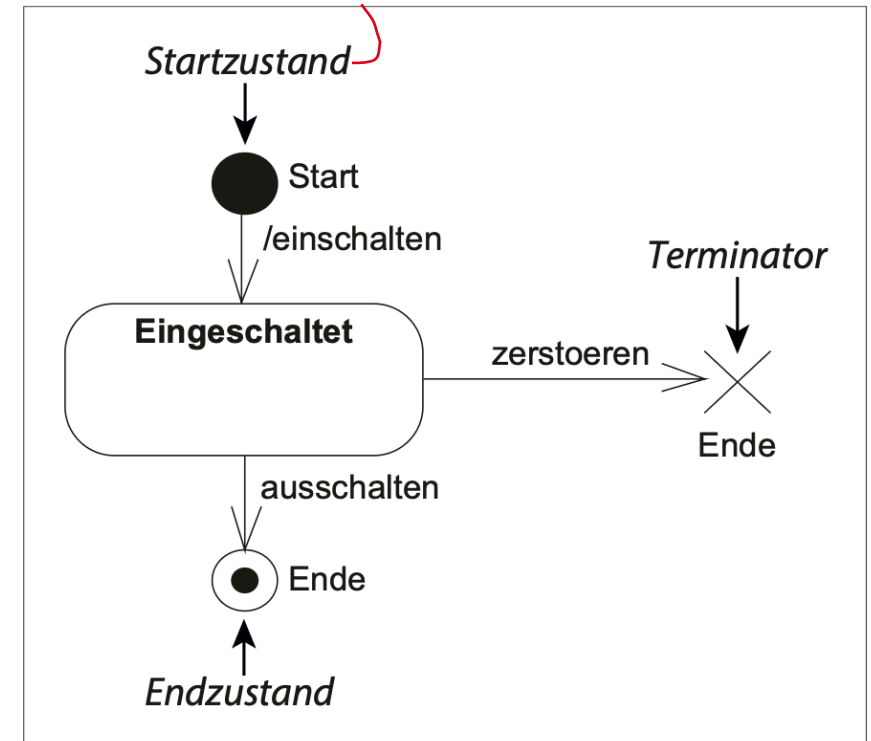
Die Ausführung eines **ganzen Zustandsautomaten** ist beim Erreichen eines **Terminators** (engl. Terminator) beendet.“ – Kecher*



Start-, Endzustände und Terminatoren sind **Pseudo-Zustände**, das Objekt durchläuft sie, kann dort aber nicht verbleiben.



Meist ist nur ein Startzustand (pro Ebene) sinnvoll!



Bildquelle Kecher*

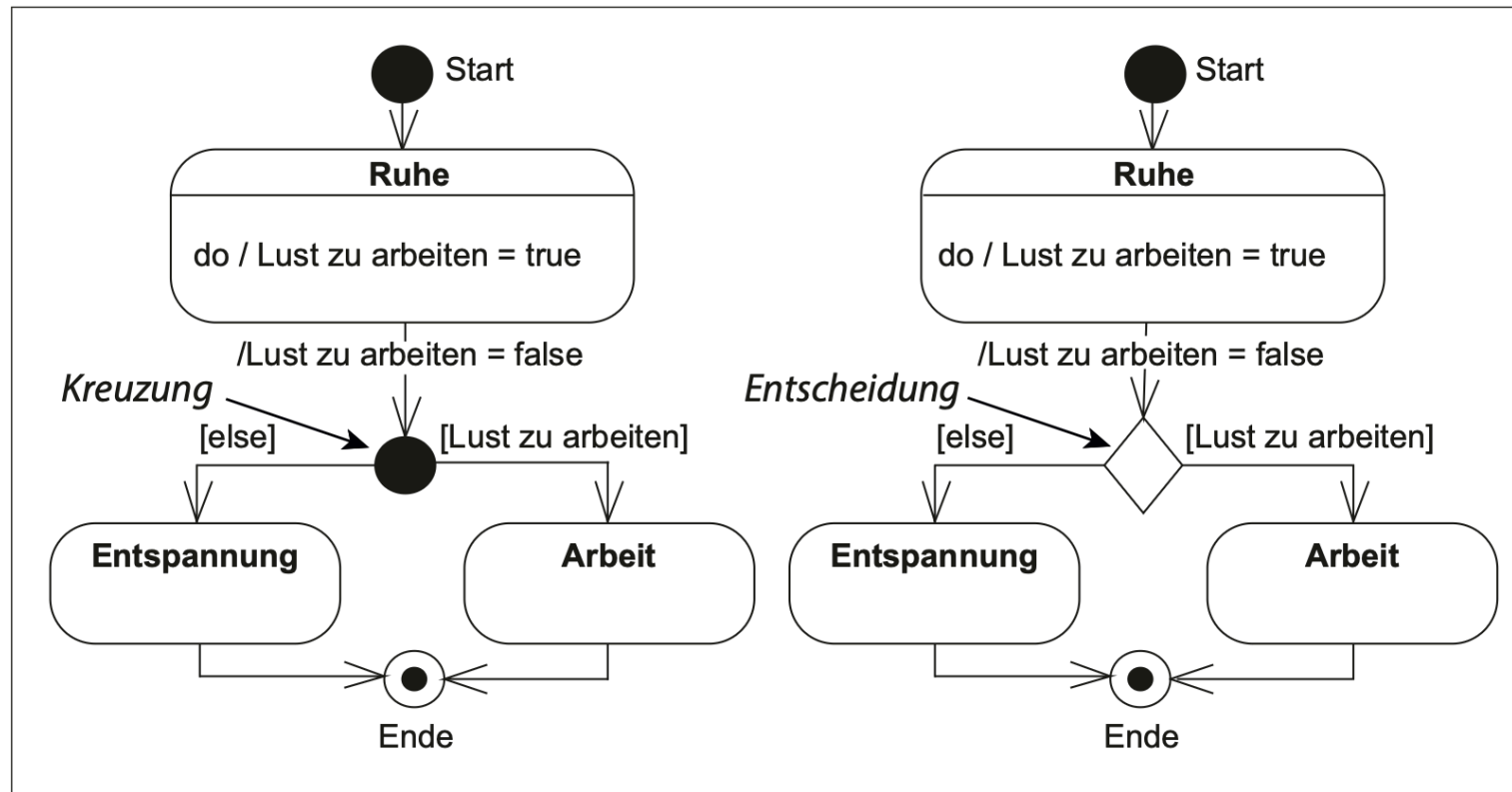
Beliebig viele
Endzustände und
Terminatoren erlaubt!

UML-NOTATIONSELEMENT: PSEUDO-ZUSTÄNDE: ENTSCHEIDUNG UND KREUZUNG

Entscheidung und Kreuzung

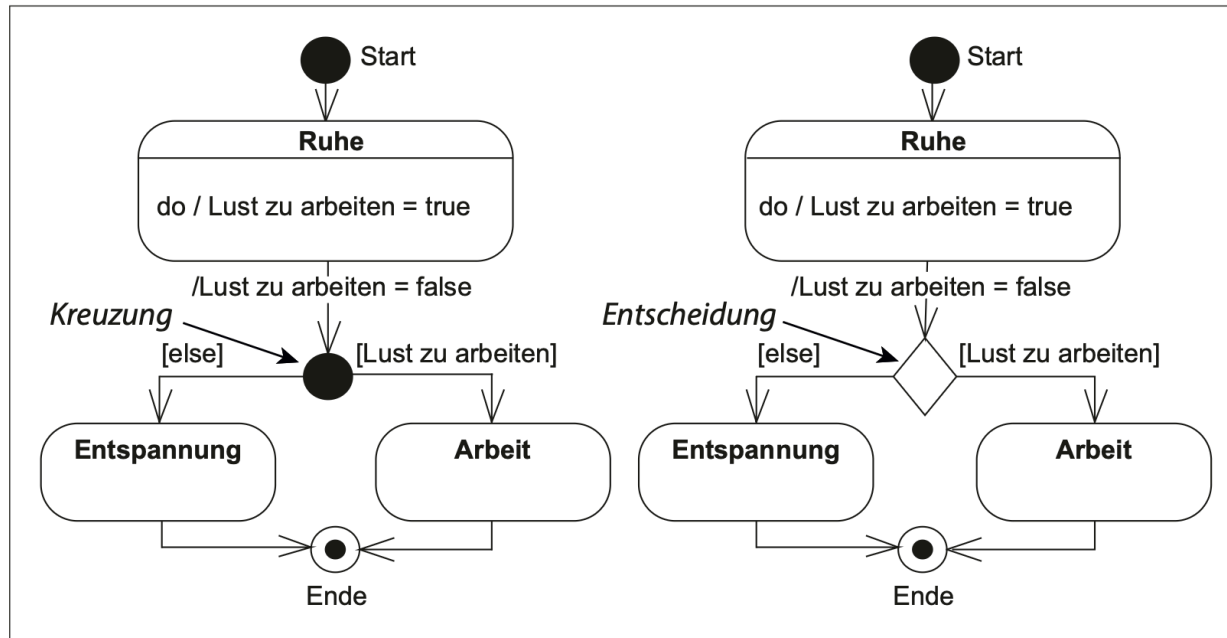


„Eine **Kreuzung** (engl. Junction) modelliert eine **Hintereinanderschaltung** von **Transitionen**. **Entscheidungen** (engl. Choices) **modellieren dynamische Verzweigungen**.“ – Kecher*



Bildquelle Kecher*

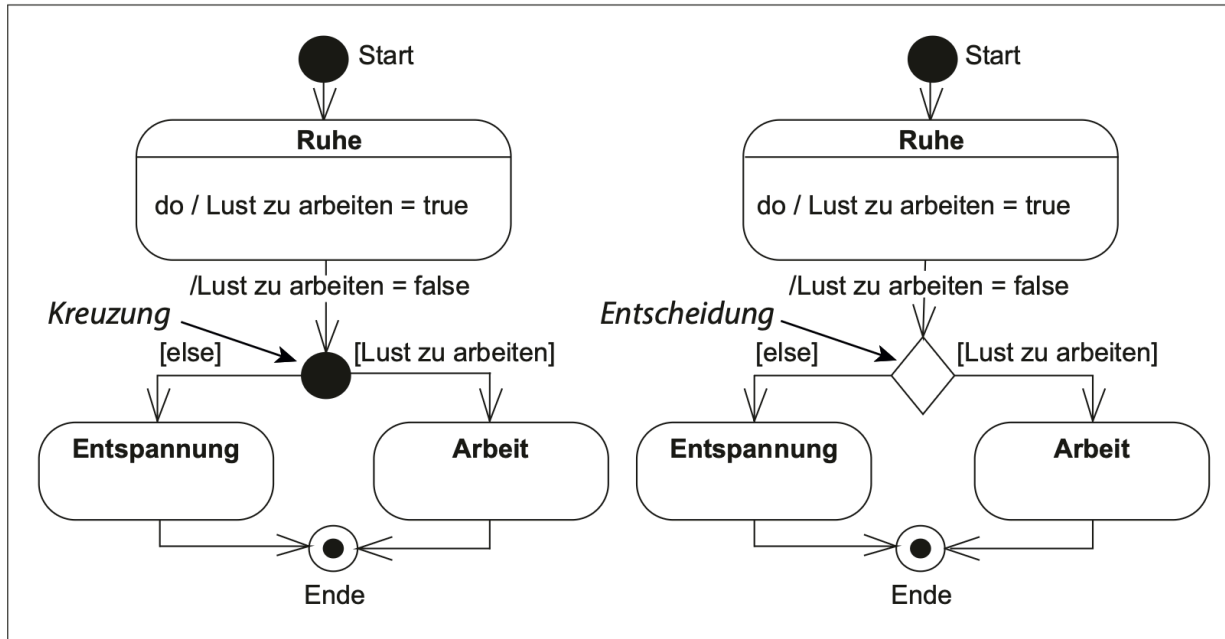
Entscheidung und Kreuzung



Bildquelle Kecher*

- Kreuzung (Junction) bündelt mehrere eingehende Transitionen zu einer ausgehenden
- Oder: Aufteilung einer Transition in mehrere mit Guards versehene Alternativen
- Die Entscheidung über den Verlauf nach der Kreuzung erfolgt vor dem Verlassen des Zustands (statisch).

Entscheidung und Kreuzung

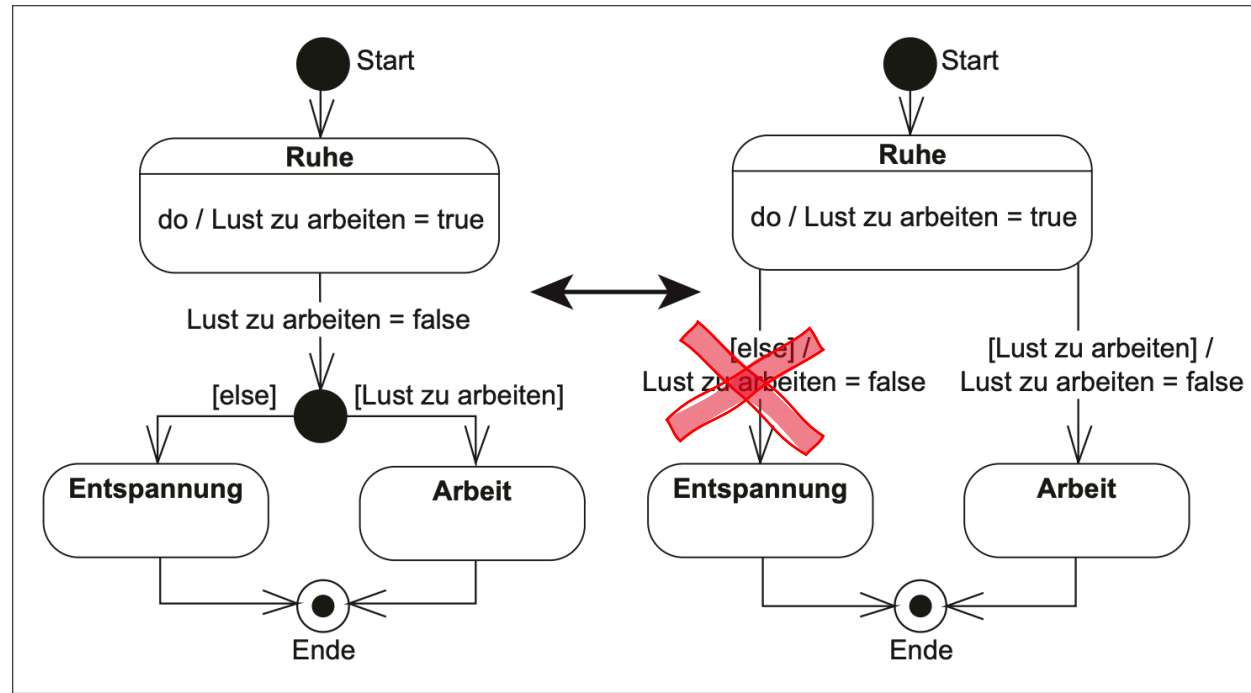


Bildquelle Kecher*

- Entscheidung (Choice) teilt eine eingehende Transition in mehrere guard-basierte Alternativen
- Auswahl erfolgt dynamisch beim Erreichen des Entscheidungsknotens
- Unterschied zur Kreuzung (Junction):
 - Kreuzung → Entscheidung auf Basis der Werte vor Beginn der Transition
 - Entscheidung → Auswahl auf Basis der Werte beim Erreichen des Entscheidungsknotens innerhalb der Transition
- Choice: Guards werden ausgewertet, nachdem die Aktionen bis zum Entscheidungsknoten bereits ausgeführt wurden.
- Folge: Ergebnis kann sich durch zwischenzeitliche Aktionen unterscheiden

*Quelle: UML 2.5 - Das umfassende Handbuch, Kecher, Hoffmann-Elbern, Will, Rheinwerk, 2021

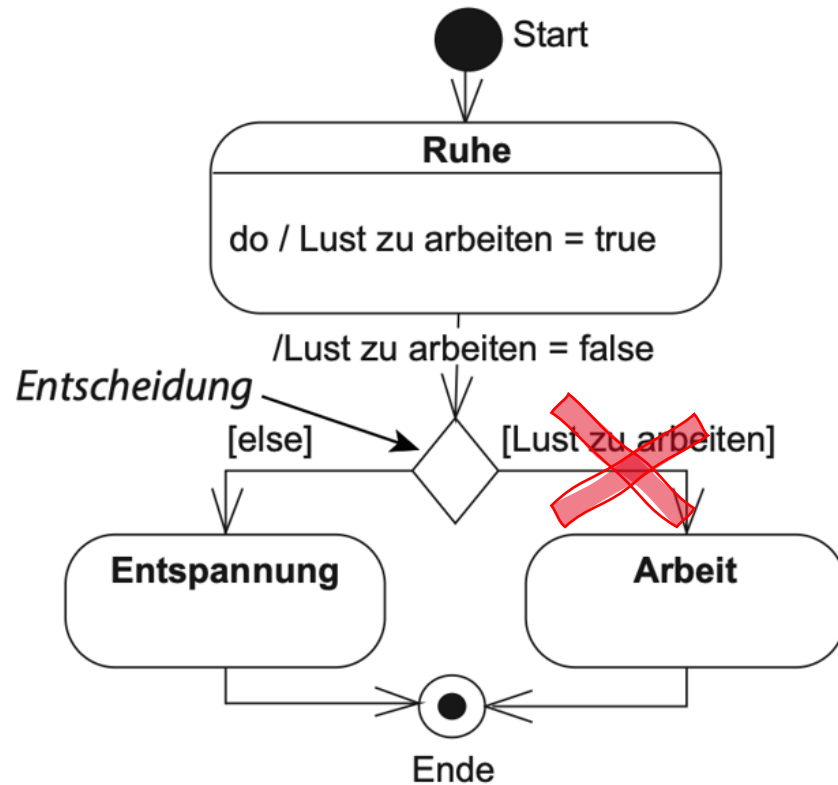
Ersetzung der Kreuzung durch äquivalente Transitionen



Bildquelle Kecher*

- Bei Kreuzungen (statisch) erfolgt die Auswahl bereits beim Verlassen des Zustands
- Guards werden mit den aktuellen Attributwerten des aktiven Zustands vor der Transition bewertet.
- Dadurch können Kreuzungen durch direkte, äquivalente Transitionen ersetzt werden
- Vorteil: Diagramm wird einfacher und übersichtlicher
- Im Beispiel kann der else-Fall nicht zur Ausführung kommen.

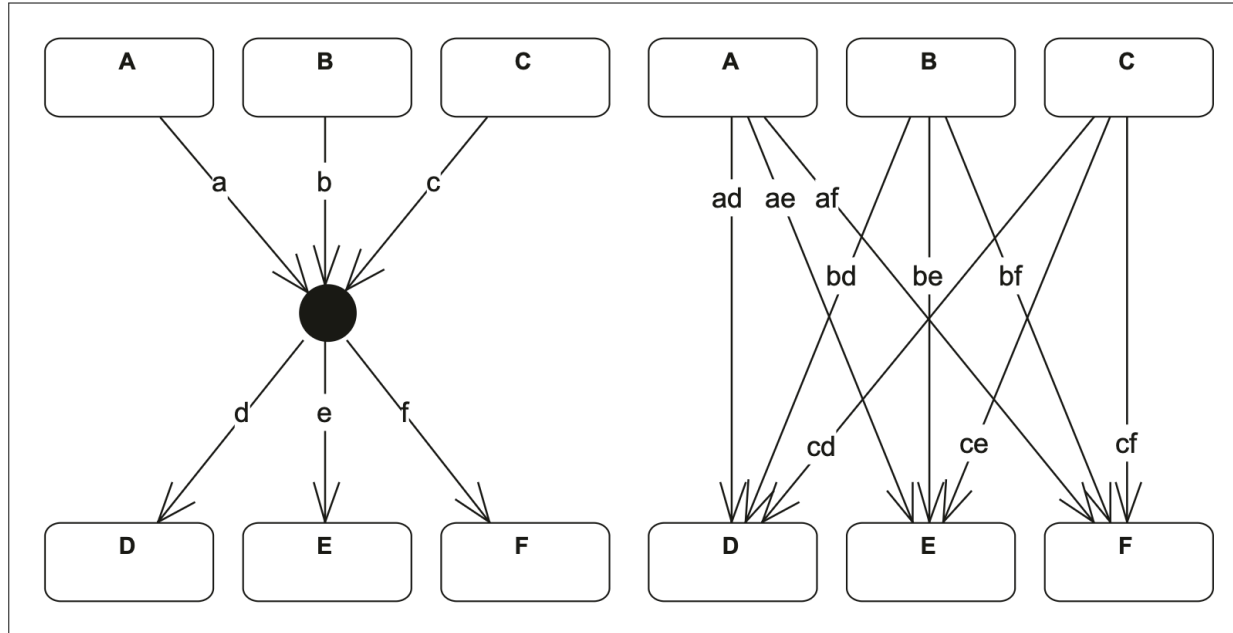
Entscheidung und Kreuzung



Bildquelle Kecher*

- Entscheidung (Choice): Auswahl erfolgt dynamisch beim Erreichen des Entscheidungsknotens
- Falls die gezeigten Zustände **nicht Teil** einer Region oder eines zusammengesetzten Zustands ($\rightarrow$ später) sind, kann der true-Fall nicht eintreten.
- Wenn doch, hängt es an der genauen Reihenfolge, die sich aus der Run-to-Completion-Semantik ergibt.
- Wird unübersichtlich: schlechter Modellierungsstil!

Ersetzung der Kreuzung durch äquivalente Transitionen



Bildquelle Kecher*

- Kreuzungen (Junctions) können zur Zusammenführung und Verzweigung von Transitionen genutzt werden
- Vereinfachen Modelle bei vielen sich kreuzenden Transitionen
- Erhöhen die Übersichtlichkeit und Lesbarkeit
- Alternative ohne Kreuzung ist inhaltlich gleich, aber oft unübersichtlicher

*Quelle: UML 2.5 - Das umfassende Handbuch, Kecher, Hoffmann-Elbern, Will, Rheinwerk, 2021

Aufgabe 2: Welche Aussagen treffen zu?

A

Nach der Transition `login()` wird bereits vor dem Knoten nach „Prüfung läuft“ entschieden, ob in den Zustand „Eingeloggt“ oder „Konto gesperrt“ gewechselt wird.

B

Die Guards `[Session gültig]` und `[else]` werden erst am Entscheidungsknoten nach „Eingeloggt“ ausgewertet.

C

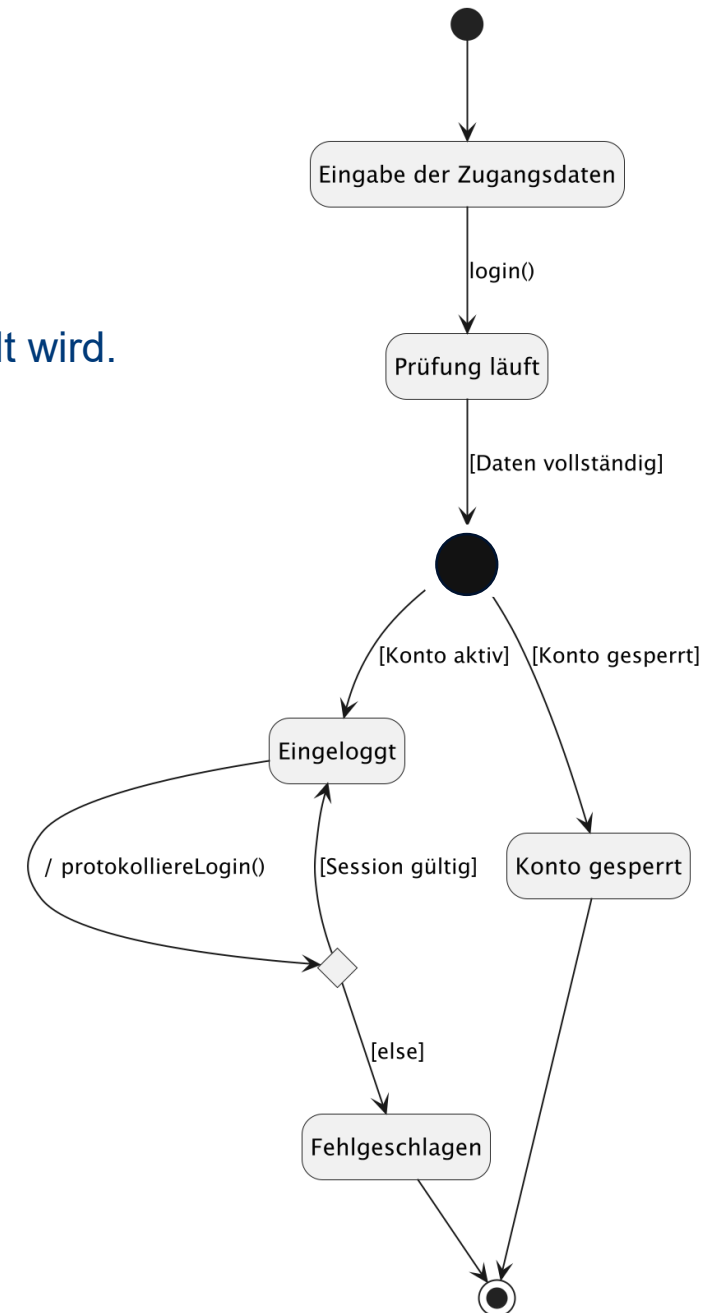
Der Knoten nach „Prüfung läuft“ ist eine Entscheidung und modelliert daher eine dynamische Verzweigung.

D

Am Entscheidungsknoten nach „Eingeloggt“ wird abhängig von den Guards genau eine ausgehende Transition gewählt.

E

Am Knoten nach „Prüfung läuft“ werden die Guards erst beim Erreichen des Knotens ausgewertet.





ÜBUNGSaufGABE 6.2

Zustandsdiagramm Online-Prüfung

Übungsaufgabe 6.2: Zustandsdiagramm Online-Prüfung

Modellieren Sie ein Zustandsdiagramm für den Ablauf einer Online-Prüfung.

Berücksichtigen Sie folgende Anforderungen:

- Eine Prüfung beginnt im Zustand Nicht gestartet
- Nach Start durch Studierende wechselt sie in den Zustand In Bearbeitung
- Beim Eintritt in den Zustand In Bearbeitung wird die Aktion timerStarten() ausgeführt
- Während der Bearbeitung wird kontinuierlich an der Prüfung gearbeitet
- Nach Abgabe wird die Prüfung bewertet
- Abhängig vom Ergebnis wird entschieden:
 - ≥ 50 Punkte $\rightarrow$ Zustand Bestanden
 - < 50 Punkte $\rightarrow$ Zustand Nicht bestanden
- Diskutieren Sie drei Varianten für die Modellierung der Entscheidung

UML-NOTATIONSELEMENT: ZUSAMMENGESETZTER ZUSTAND

Zusammengesetzter Zustand



Zusammengesetzte Zustände (Composite States) modellieren **Hierarchien** von **Zuständen**. Ein Zustand kann dabei **weitere Unterzustände** enthalten. Sie dienen zur **Strukturierung komplexer Zustandsdiagramme**.

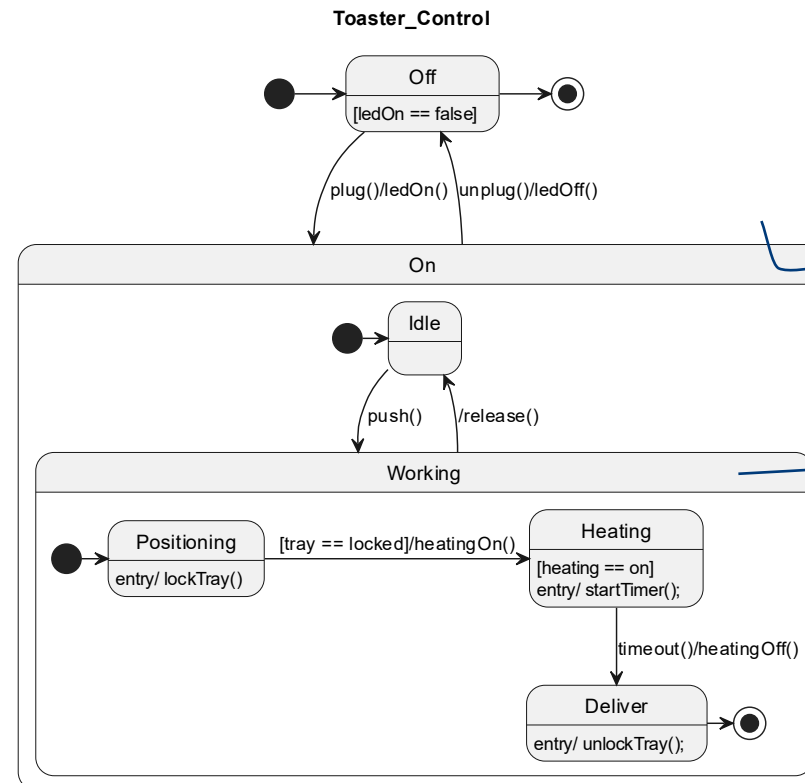
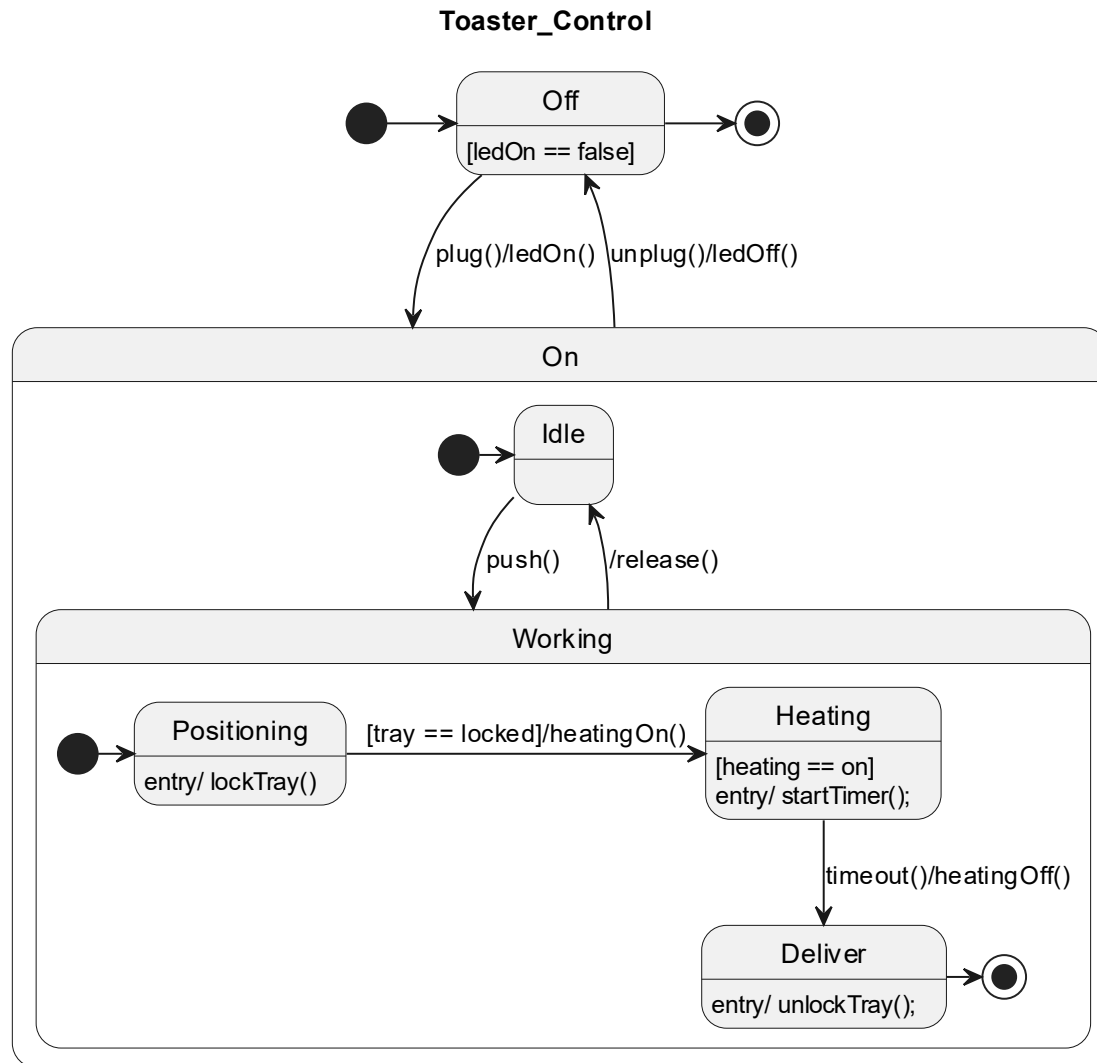
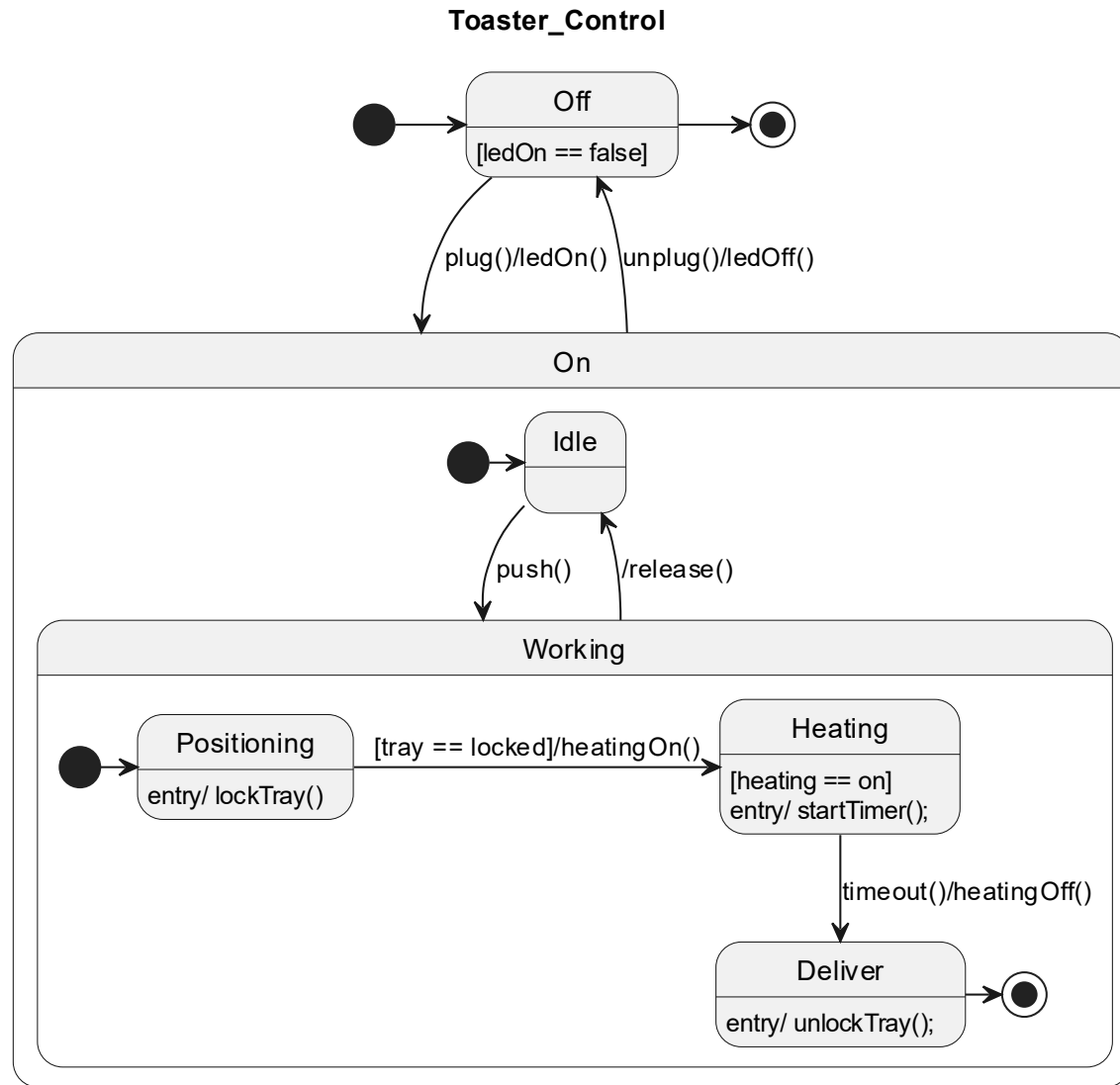


Diagramm mit zwei zu
Zusammengesetzten
Zuständen



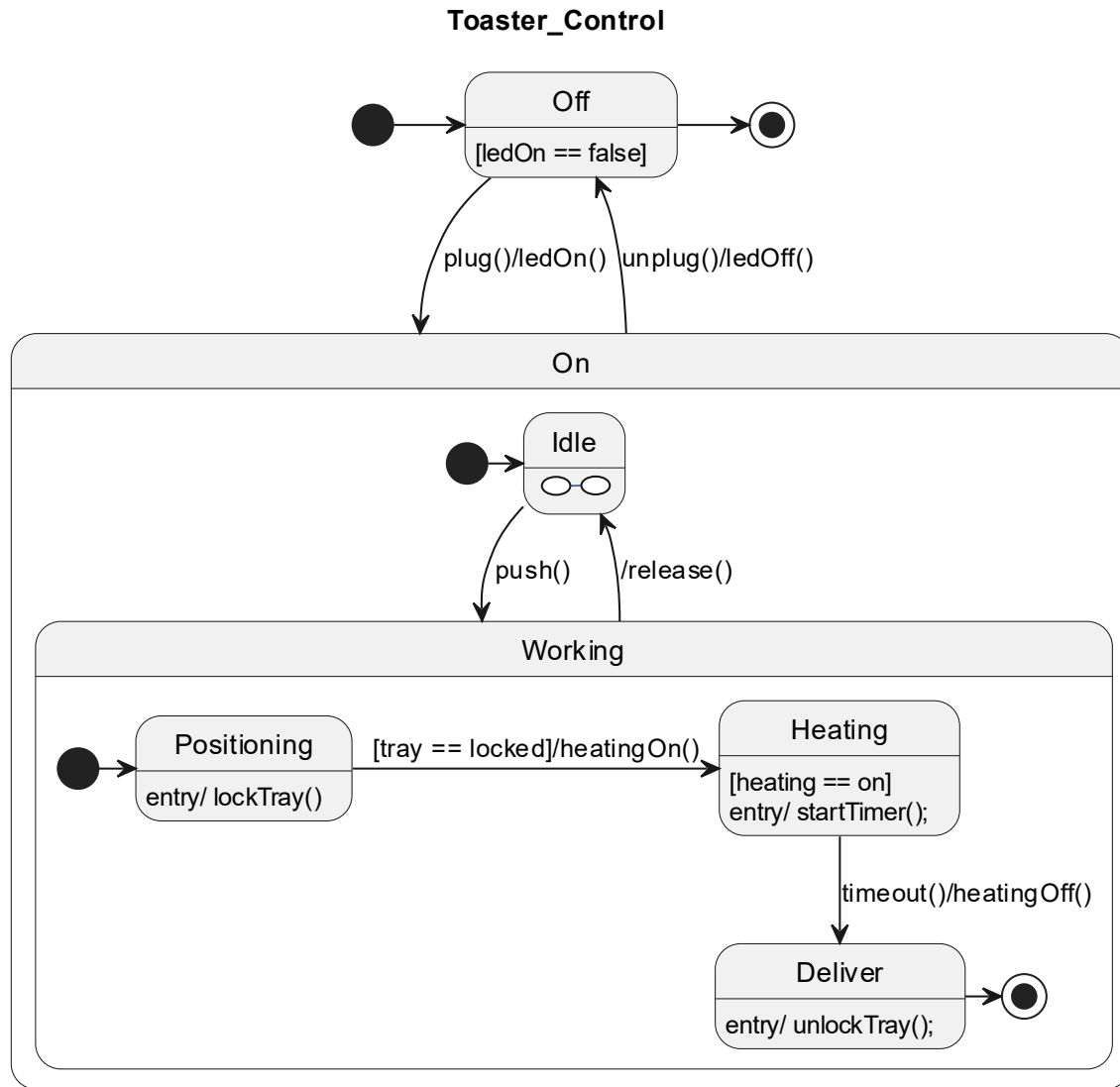
Zustand On:

- Enthält Unterzustände: Idle und Working
- Bei Transition von **Off** nach **On** → **zusammengesetzte Transition** mit Eintritt in Idle über Startzustand
 - Erinnerung: in Pseudostates kann sich ein Objekt nicht aufhalten.



Zustand Working

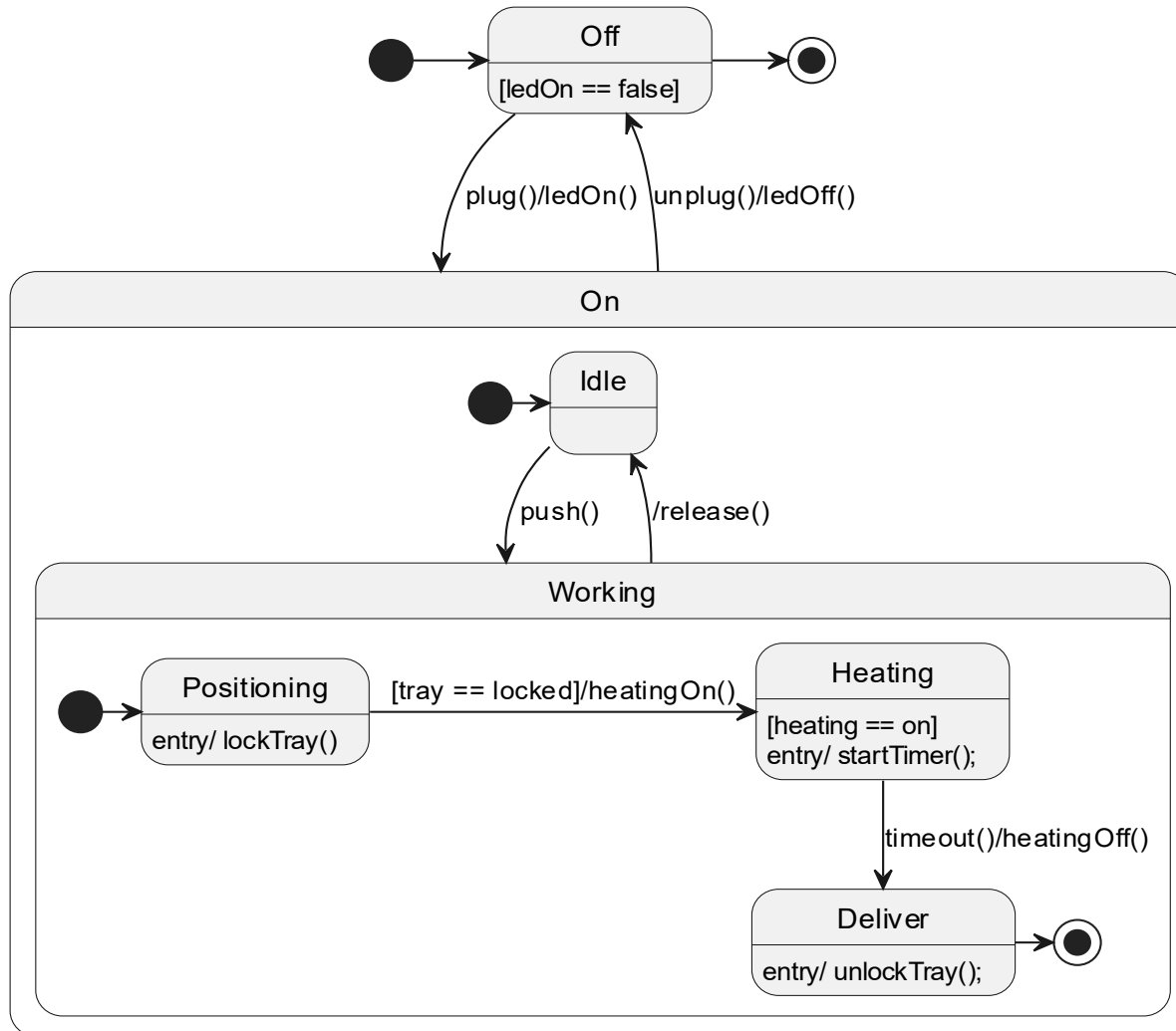
- Enthält Unterzustände: Positioning, Heating und Deliver
- Bei Transition von **Idle** nach **Working** → **zusammengesetzte Transition** mit Eintritt in Positioning über Startzustand
- Wechsel zu Heating, sobald `tray == locked`
- Wechsel zu Deliver bei `timeout()`
- Deliver → Beendet Working als den aktiven Zustand, es folgt zusammengesetzte Transition zu Idle über Endzustand und mit `release()`



Zustand Idle

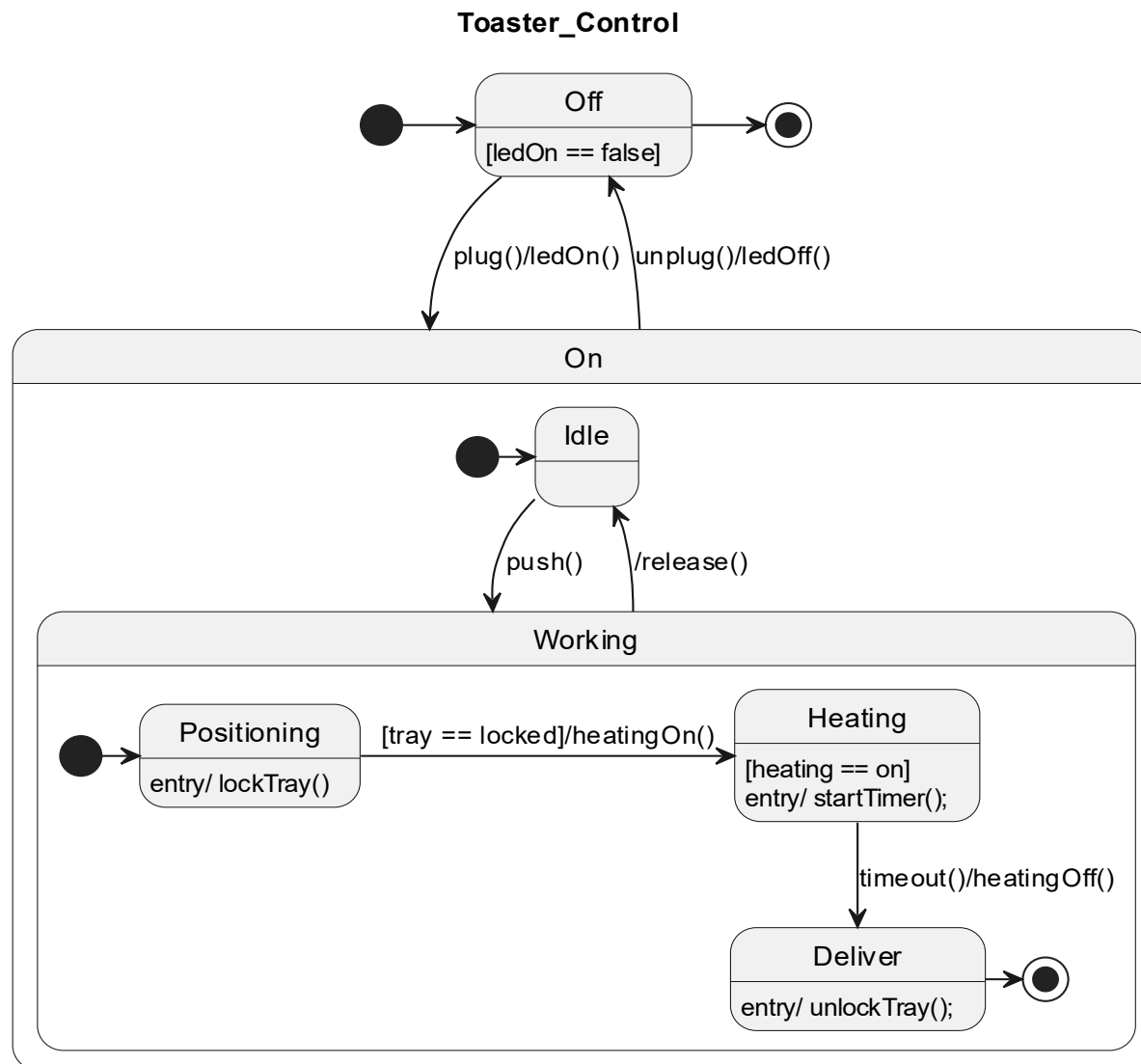
- jetzt: Zusammengesetzter Zustand: ○○
- Innere Zustände hier ausgeblendet („versteckte Dekomposition“)
- Wechsel zu Working mit push()

Toaster_Control



Hierarchie von Zuständen

- Ist ein zusammengesetzter Zustand aktiv, ist mindestens einer seiner Unterzustände aktiv.
- Beispiel:
 - **Heating** und **Working** und **On** sind aktiv (gleichzeitig)



Austritt bei konkurrierenden Transitionen

Es können verschiedene Transitionen aktiviert sein, auf verschiedenen Zustandsebenen.

Beispiel:

Heating und Positioning und On sind aktiv

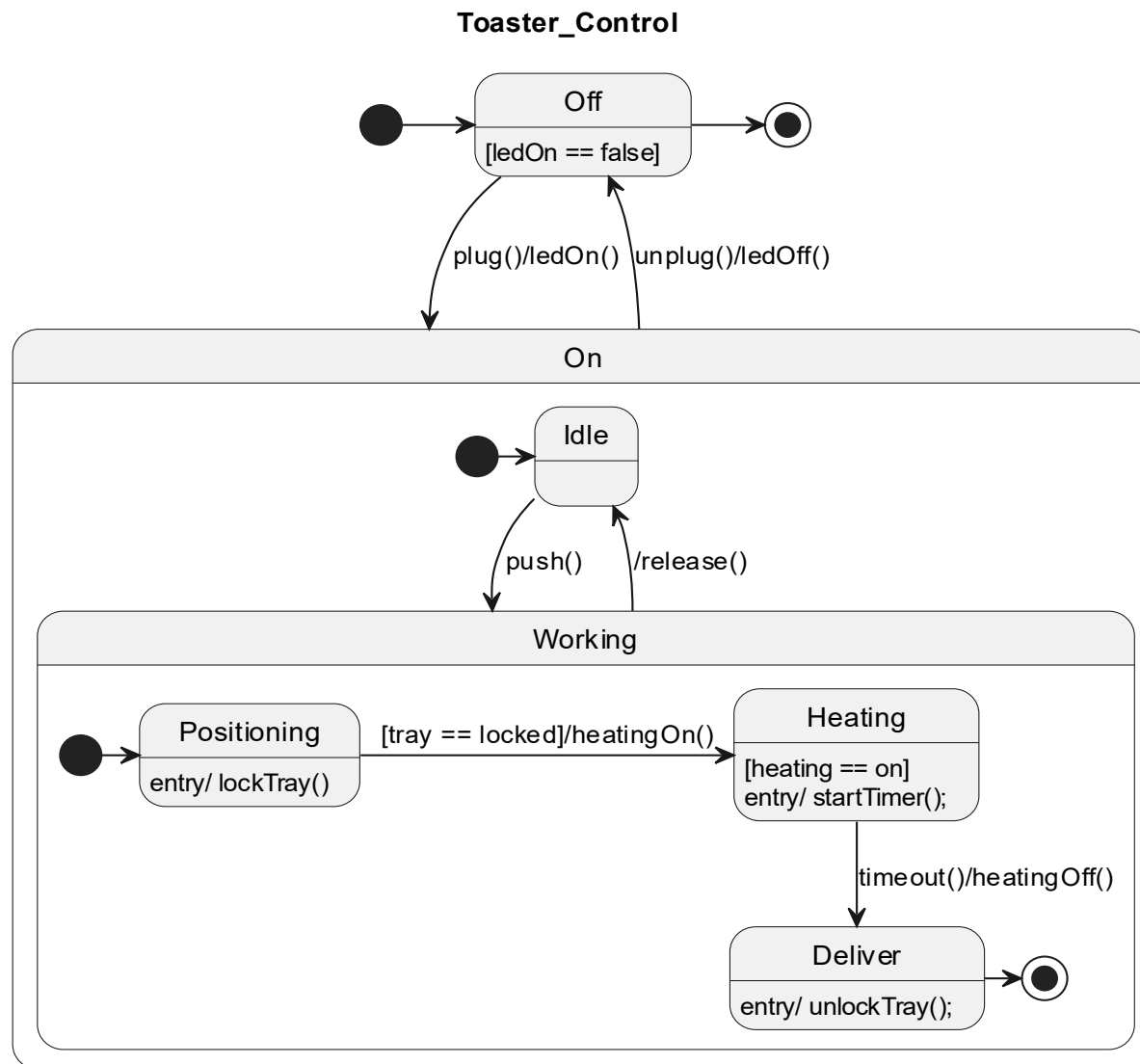
- `plug()` tritt auf und `[tray == locked]` gilt, was nun?

UML Semantik:

- Eine aktivierte Transition, die aus einem aktiven Oberzustand herausführt, hat **Vorrang** vor einer aktivierten Transition eines seiner aktiven Unterzustände.

Beispiel:

Transition `plug()`: **On** -> **Off** hat Vorrang, alle Unterzustände von **On** werden verlassen (Abbruch)



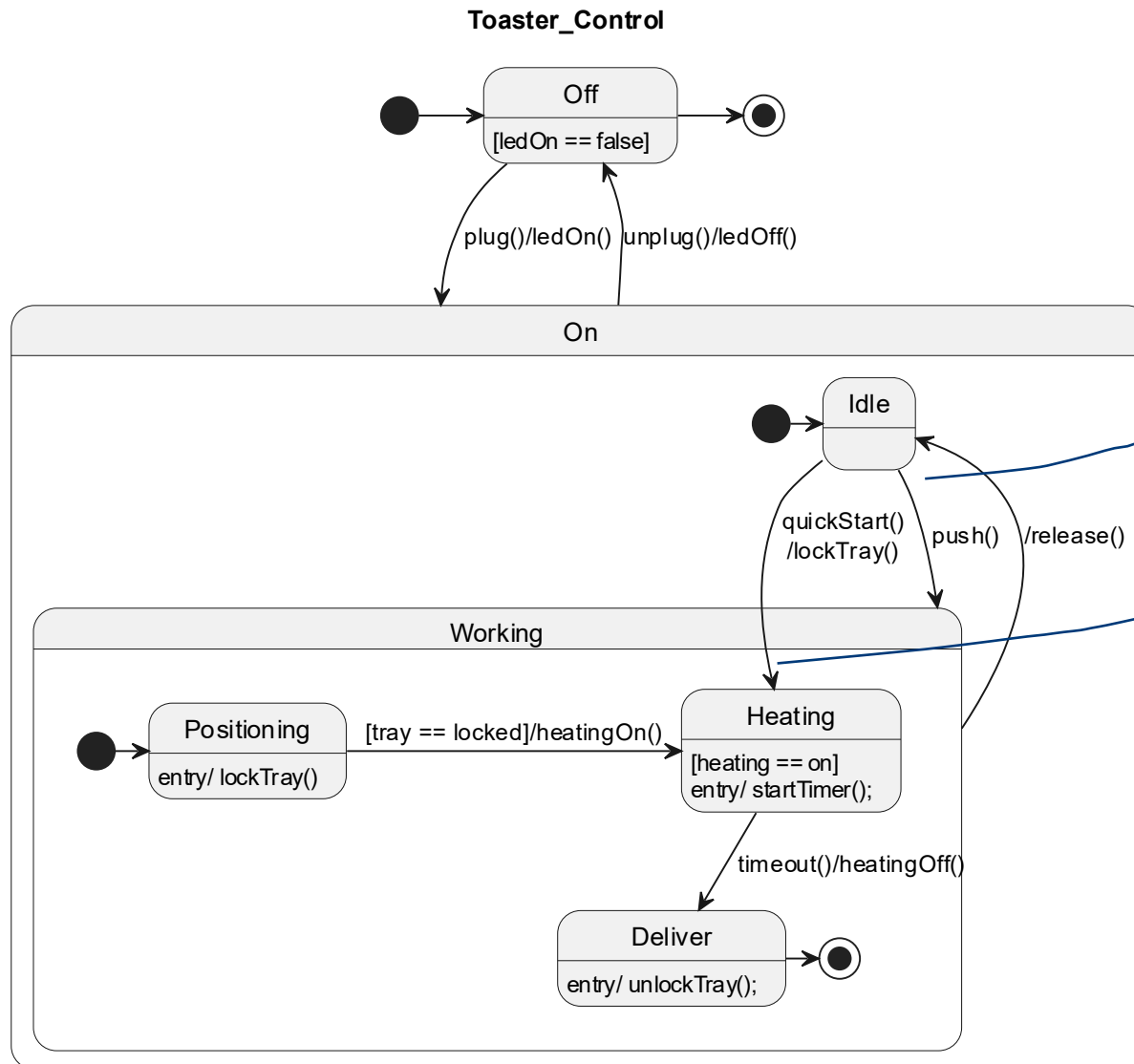
Austritt bei konkurrierenden Transitionen

UML Semantik:

- Eine aktivierte Transition, die aus einem aktiven Oberzustand herausführt, hat **Vorrang** vor einer aktivierten Transition eines seiner aktiven Unterzustände. (**outermost-firing**)

Hinweis zu SysML (insb. für eingebettete Systeme):

- Bei hardwarenahen Modellen kann es sinnvoll sein, die Austrittsregel umzukehren (innermost-firing), die innere Komponente führt aktivierte Transitionen noch aus, bevor ein Oberzustand verlassen werden kann.

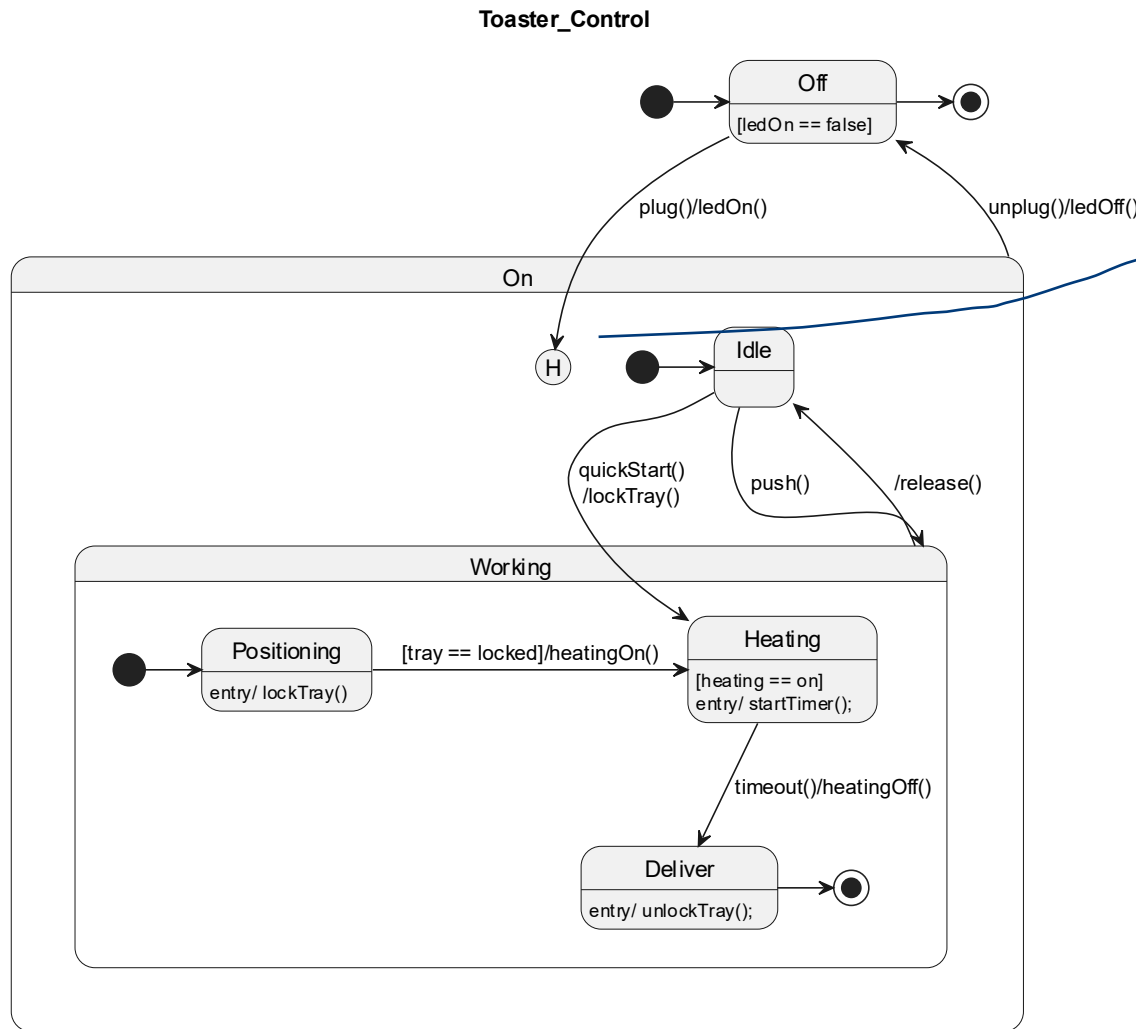


Standard-Eintritt (Default Entry)

- Transition endet am Rand eines zusammengesetzten Zustands
- → Startzustand wird automatisch aktiviert

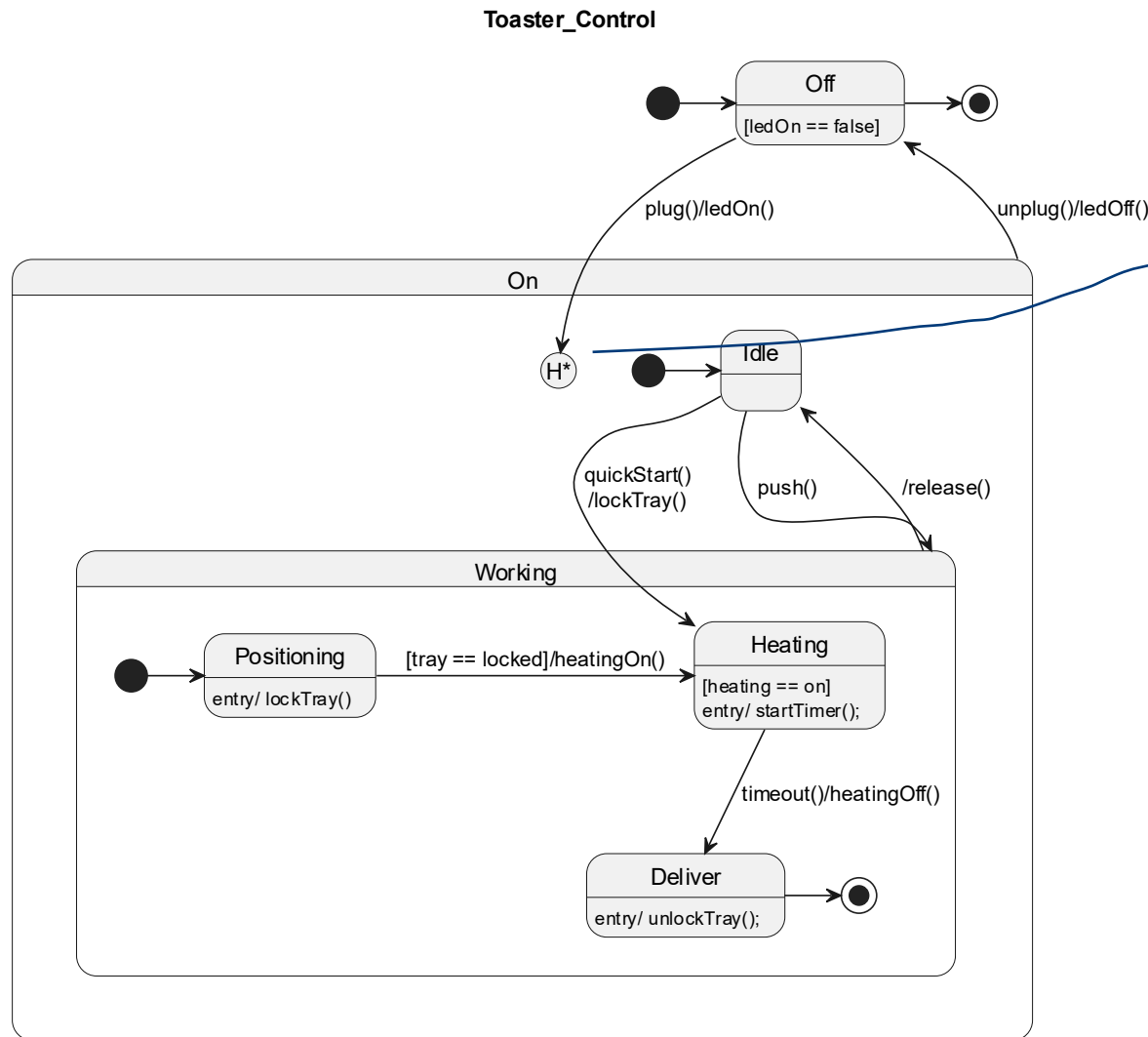
Expliziter Eintritt (Explicit Entry)

- Transition führt direkt zu einem Unterzustand
- → dieser Unterzustand wird betreten.



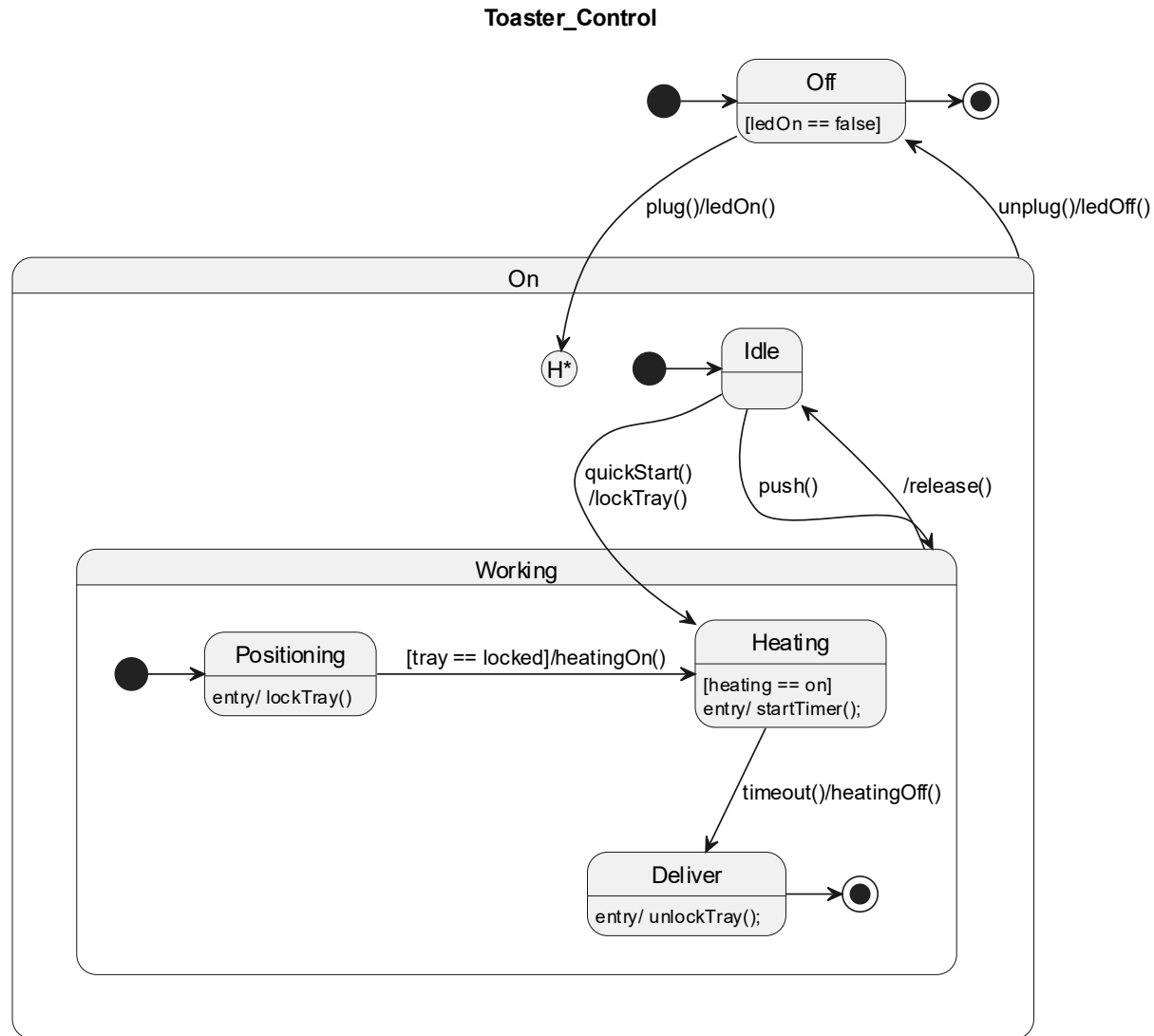
Eintritt über die flache Historie
(Shallow History Entry) (H)

- Merkt sich den letzten aktiven Unterzustand (oberste Ebene)
- Beim Wiedereintritt → Rückkehr dorthin
- Beim ersten Eintritt → definierter Standardpfad
- History ist ein Pseudostate



Eintritt über die tiefe Historie (Deep History Entry) (H*)

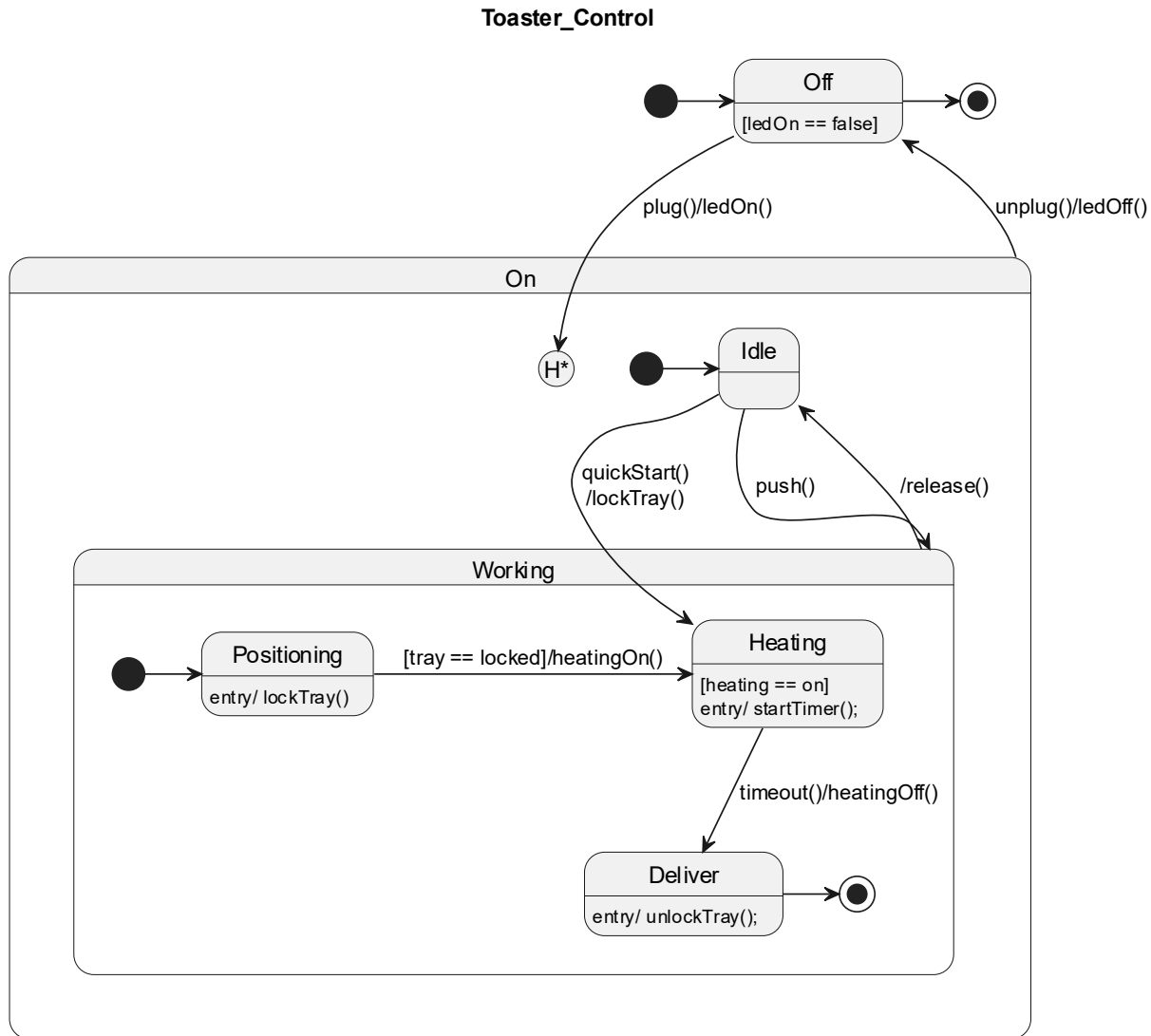
- Merkt sich den letzten aktiven Unterzustand über alle Ebenen
- → Rückkehr in tiefste zuvor aktive Struktur
- Beim ersten Eintritt → definierter Standardpfad
- Deep-History ist ein Pseudostate



Historie = „Gedächtnis“ des Zustandsautomaten

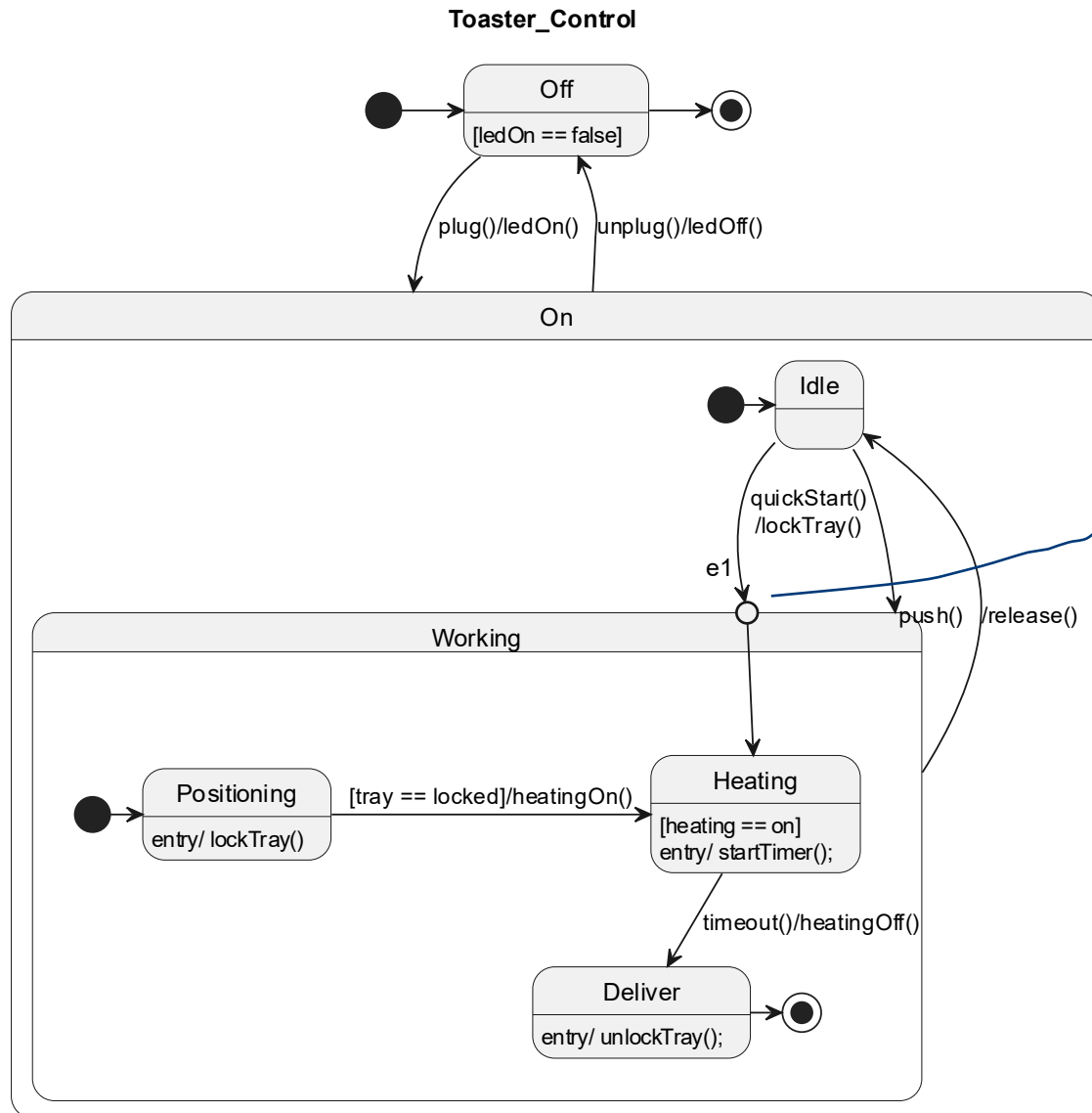
Unterschied:

- Ⓜ **H** ■ flach → nur oberste Ebene, in der History-Operator verzeichnet ist
- Ⓜ **H*** ■ tief → über gesamte Hierarchie



Vorsicht: Gedächtnis kostet Speicher.

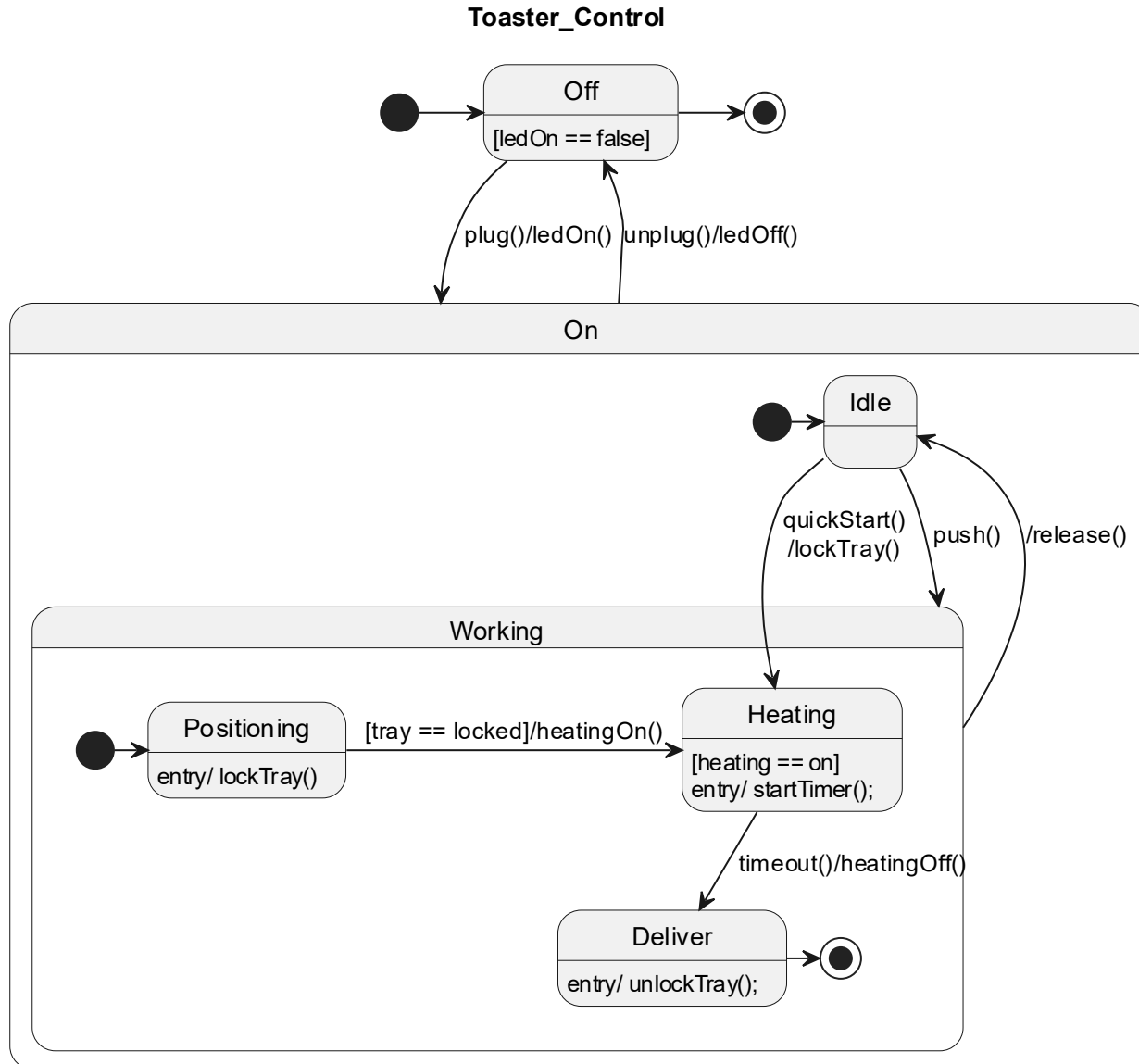
Ein History-Operator bedeutet, dass sich bei Verlassen der Zustand gemerkt werden muss, um beim Wiedereintritt zurückkehren zu können (ggf. deep copy)



Eintrittspunkt (Entry Point)

- Transition führt zu einem benannten Eintrittspunkt
- → vordefinierter Einstieg in die Zustandsstruktur

Verwendung zusammengesetzter Zustände



- Zusammengesetzte Zustände ermöglichen unterschiedliche Abstraktionsgrade
- Diagramme können schrittweise verfeinert (detailliert) werden
- Interne Zustände können ausgeblendet werden
- Eine Transition von/zum übergeordneten Zustand ersetzt Bündel von Transitionen zu den untergeordneten Zuständen
- → unterstützt Übersicht (Zoom-Out) und Detailanalyse (Zoom-In)

Aufgabe 3: Welche Aussagen treffen zu?

A

Beim Eintritt in den Zustand „Bestellung bearbeiten“ wird zunächst der Unterzustand „Prüfung“ aktiviert.

B

Befindet sich das Objekt im Zustand „Rechnung erstellen“, dann ist gleichzeitig auch der Zustand „Bestellung bearbeiten“ aktiv.

C

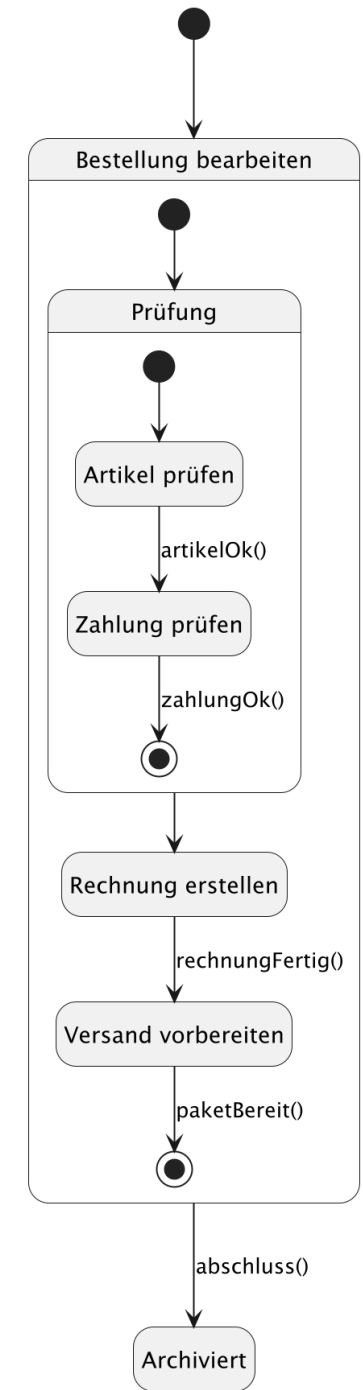
Die Zustände „Artikel prüfen“ und „Rechnung erstellen“ können gleichzeitig aktiv sein.

D

Mit dem Event `abschluss()` wird der zusammengesetzte Zustand „Bestellung bearbeiten“ verlassen.

E

Der Zustand „Prüfung“ ist selbst wieder ein zusammengesetzter Zustand.





ÜBUNGSaufGABE 6.3

Zustandsdiagramm Smartphone-Anruf

Übungsaufgabe 6.3: Zustandsdiagramm Smartphone-Anruf

Modellieren Sie ein Zustandsdiagramm für den Ablauf eines Telefonanrufs auf einem Smartphone.

Das Smartphone befindet sich zunächst im Zustand Bereit. Trifft ein eingehender Anruf ein, wechselt es in den zusammengesetzten Zustand Anruf bearbeiten.

Innerhalb dieses Zustands beginnt der Ablauf im Zustand Klingeln. Der/die Benutzer/-in kann den Anruf entweder annehmen, wodurch in den Zustand Gespräch gewechselt wird, oder ablehnen, wodurch direkt der Zustand Beendet erreicht wird.

Im Zustand Gespräch findet eine laufende Aktivität (Telefonieren) statt. Durch das Event auflegen wird das Gespräch beendet und der Zustand Beendet erreicht.

Nach dem Zustand Beendet wird der zusammengesetzte Zustand verlassen, und das Smartphone kehrt in den Zustand Bereit zurück.

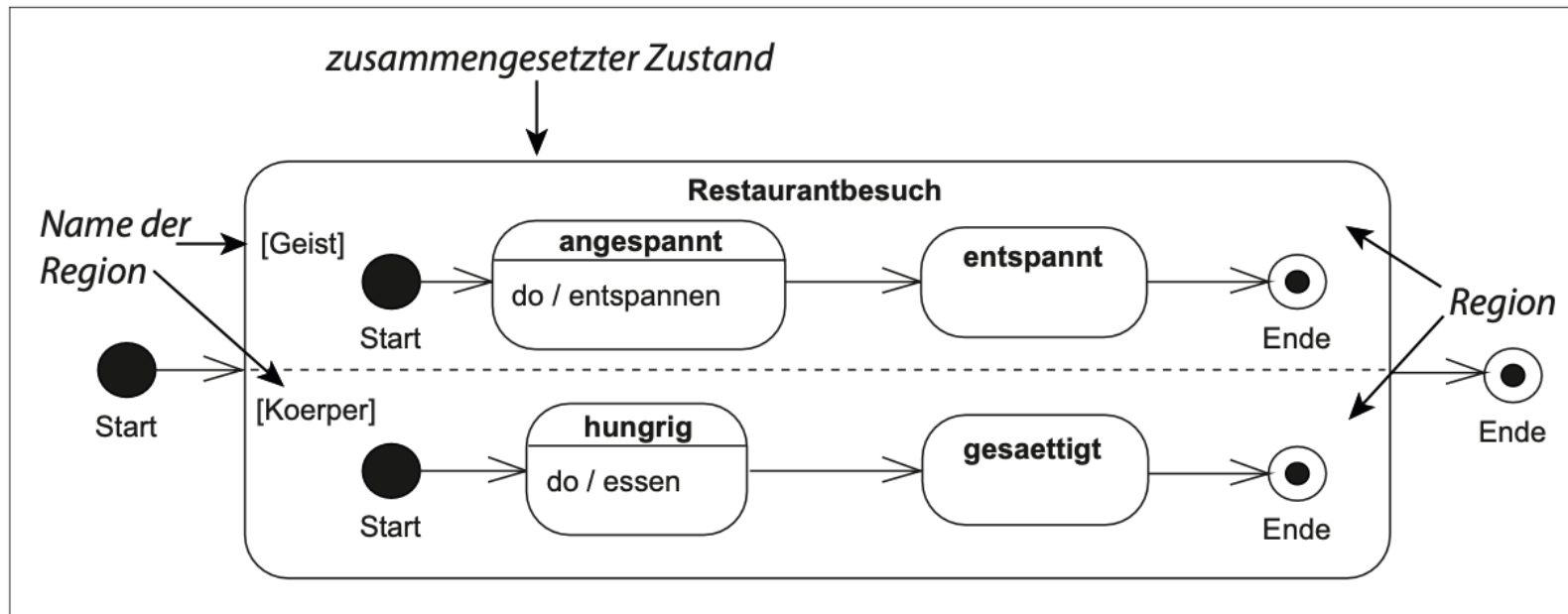
Modellieren Sie den Ablauf unter Verwendung eines zusammengesetzten Zustands sowie geeigneter Start- und Endzustände innerhalb dieses Zustands.

UML-NOTATIONSELEMENT: REGION

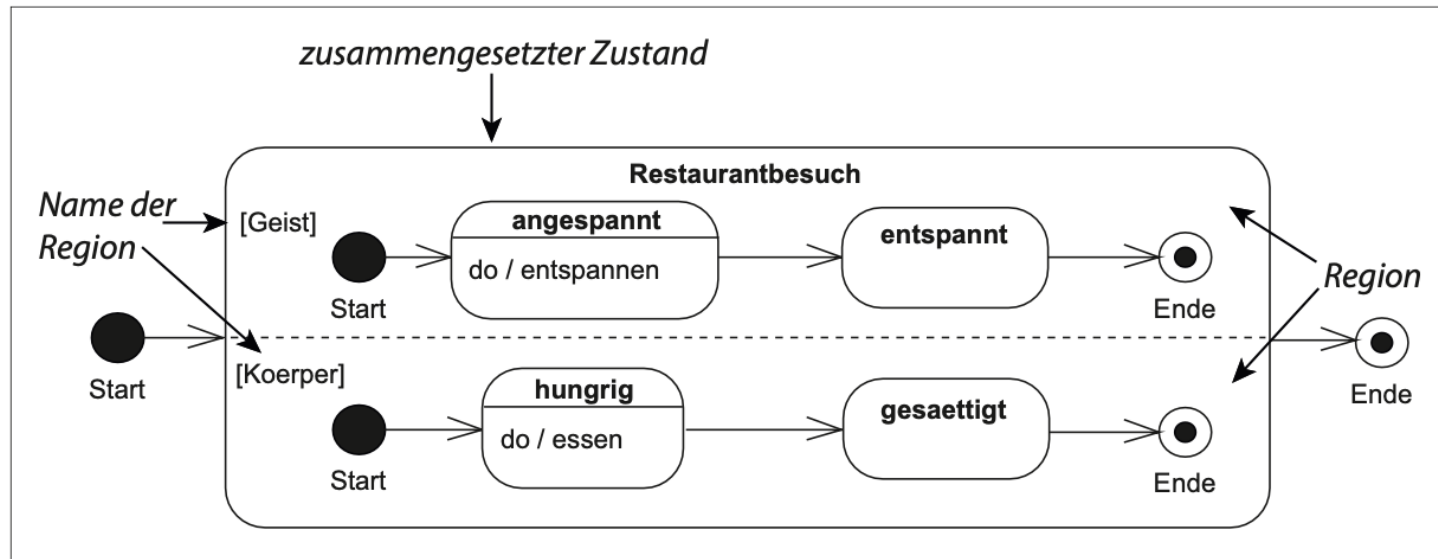
Regionen (Parallelität in Zustandsdiagrammen)



Regionen (Regions) teilen zusammengesetzte Zustände in **disjunkte** (unabhängige) **Teilbereiche** auf. Jeder **Teilbereich** modelliert einen **eigenständigen Ablauf**.
Regionen sind Grundlage für **Parallelität** im Zustandsdiagramm.



Bildquelle Kecher*



Bildquelle Kecher*



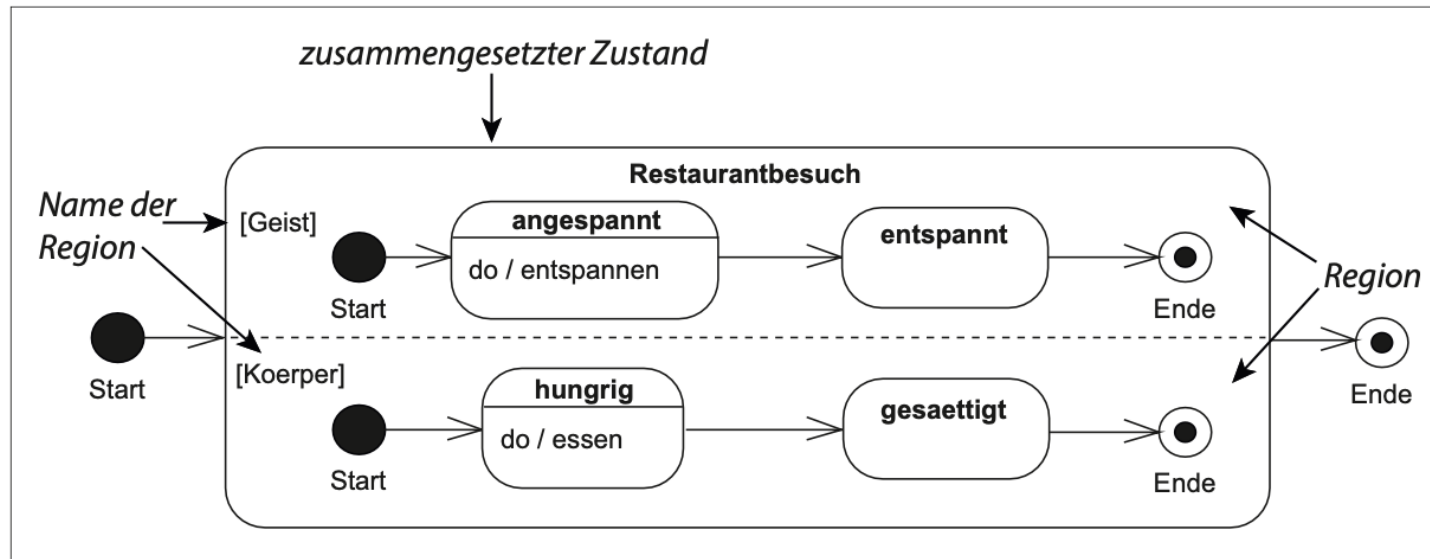
Keine Transitionen zwischen
Regionen erlaubt

Jede Region hat:

- eigenen Startzustand
- einen oder mehrere Endzustände

Beim Eintritt:

- werden alle Startzustände gleichzeitig aktiv
- → nebenläufige (parallele) Ausführung



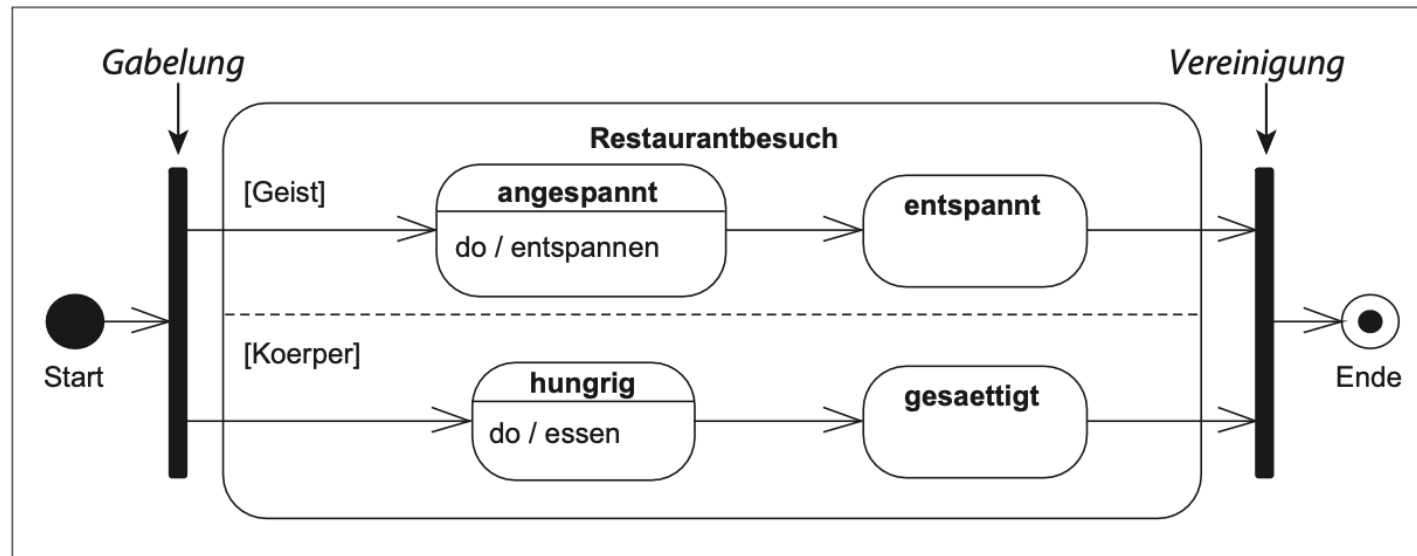
Bildquelle Kecher*

Beenden von Regionen

- Endzustand beendet nur die jeweilige Region
- Gesamter Zustand wird erst verlassen, wenn alle Regionen beendet sind
- **Ausnahme:**

Terminator in einer Region beendet gesamten Zustandsautomaten

Betreten u. Verlassen von Regionen mithilfe von Gabelungen u. Vereinigungen



- **Fork (Gabelung):** teilt eine Transition in mehrere parallele Pfade (Regionen)
- **Join (Vereinigung):** führt mehrere parallele Pfade wieder zusammen

Bildquelle Kecher*



Gabelungen und Vereinigungen ermöglichen explizite Modellierung von Parallelität und Synchronisation

*Quelle: UML 2.5 - Das umfassende Handbuch, Kecher, Hoffmann-Elbern, Will, Rheinwerk, 2021

Aufgabe 4: Welche Aussagen treffen zu?

A

Beim Eintritt in den Zustand „Bestellung läuft“ werden beide Regionen gleichzeitig gestartet.

B

Die Transition `zahlungErfolgreich()` kann direkt den Zustand „Versand vorbereiten“ in der anderen Region auslösen.

C

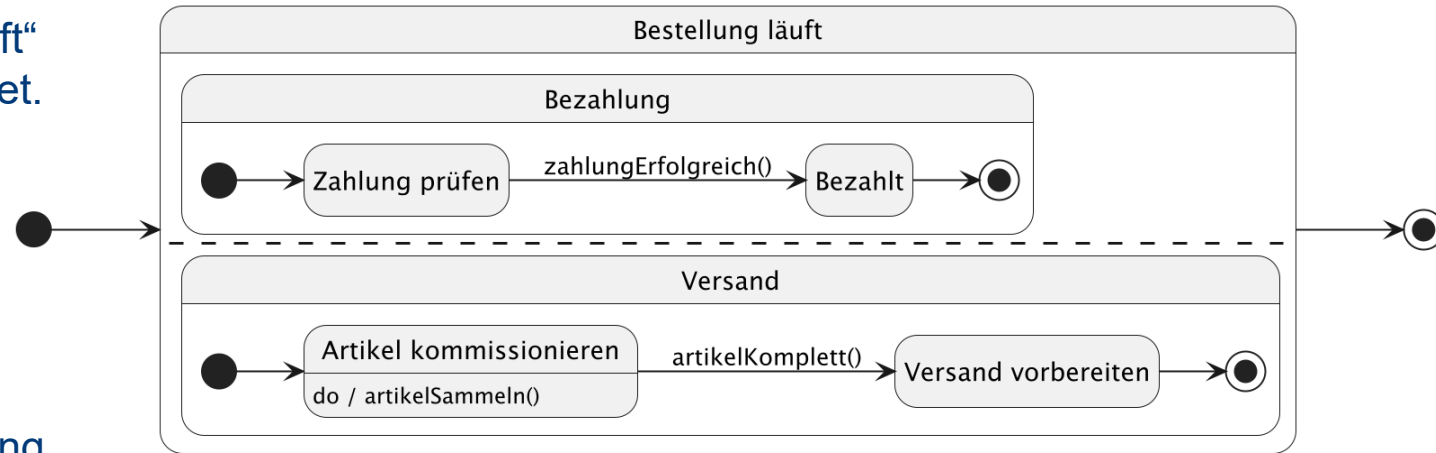
Befindet sich das Objekt im Zustand „Zahlung prüfen“, kann gleichzeitig der Zustand „Artikel kommissionieren“ aktiv sein.

D

Der Zustand „Bestellung läuft“ wird verlassen, sobald die Region Bezahlung ihren Endzustand erreicht hat.

E

In der Region Versand ist eine do-Aktion modelliert, die kontinuierlich ausgeführt wird.





ÜBUNGSaufGABE 6.4

Zustandsdiagramm Druckauftrag

Übungsaufgabe 6.4: Zustandsdiagramm Druckauftrag

Modellieren Sie ein Zustandsdiagramm für die Verarbeitung eines Druckauftrags.

Der Druckauftrag befindet sich im Zustand „Druck läuft“, der als zusammengesetzter Zustand mit zwei Regionen modelliert werden soll. Die beiden Regionen beschreiben unabhängige, parallel ablaufende Aspekte des Systems.

In der Region Druckmechanik beginnt der Ablauf im Zustand „Papier einziehen“. Anschließend wird in den Zustand „Drucken“ gewechselt und danach in den Zustand „Papier ausgeben“. Nach Abschluss dieses Ablaufs endet die Region.

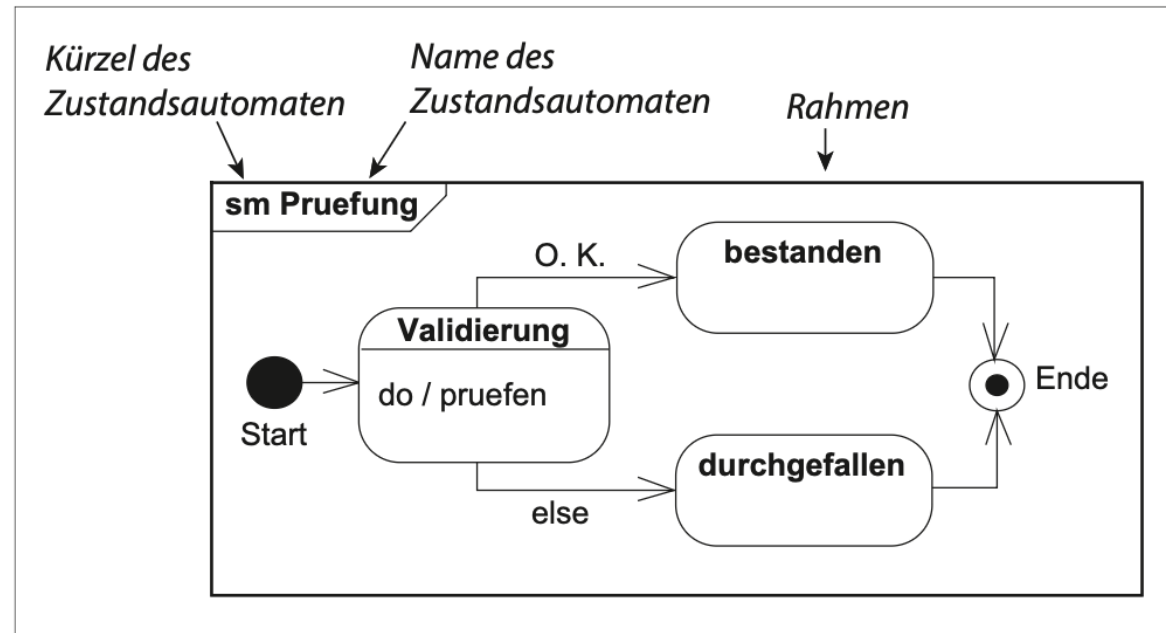
Parallel dazu wird in der Region Tintenüberwachung nach Eintritt der Zustand „Überwachen“ aktiviert. In diesem Zustand wird kontinuierlich der Tintenstand überprüft. Sinkt der Tintenstand unter einen Grenzwert, wird in den Zustand „Warnung anzeigen“ gewechselt. Ist der Tintenstand wieder ausreichend, kann zurück in den Zustand „Überwachen“ gewechselt werden. Die Tintenüberwachung endet, sobald der Druckauftrag abgeschlossen ist.

Beide Regionen starten gleichzeitig beim Eintritt in den Zustand „Druck läuft“ und werden unabhängig voneinander ausgeführt. Der Zustand „Druck läuft“ wird erst verlassen, wenn beide Regionen ihre Endzustände erreicht haben.

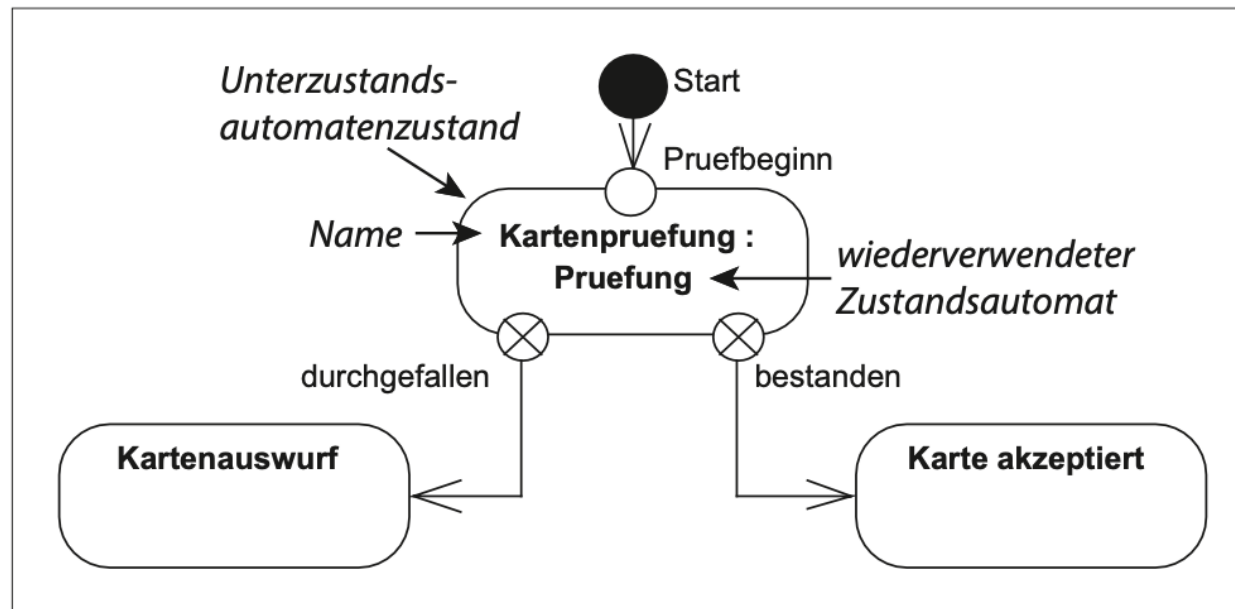
UML-NOTATIONSELEMENT: RAHMEN EINES ZUSTANDSAUTOMATEN



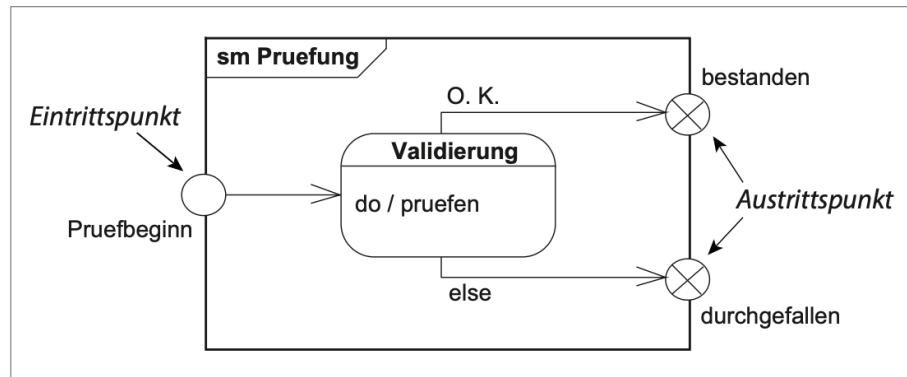
„Ein **Zustandsautomat** kann **von einem Rahmen** (engl. Frame) **umfasst** und benannt werden. Das Kürzel **sm** steht für state machine, die englische Bezeichnung für einen Zustandsautomaten.“ – Kecher*



Bildquelle Kecher*



Bildquelle Kecher*



- Wiederverwendeter Zustandsautomat wird als einfacher Zustand modelliert
- Betreten/Verlassen über Entry Points / Exit Points
- Notation: Zustandsname : Zustandsautomat
- Entspricht einer Art Instanziierung eines Zustandsautomaten
- Solche Zustände heißen Submachine States (Unterzustandsautomatenzustände)

*Quelle: UML 2.5 - Das umfassende Handbuch, Kecher, Hoffmann-Elbern, Will, Rheinwerk, 2021



ÜBUNGSaufGABE 6.5

Zustandsautomat Login-Prozess

Übungsaufgabe 6.5: Zustandsautomat Login-Prozess

Modellieren Sie den Login-Prozess für ein Online-Banking-System als Zustandsautomat.

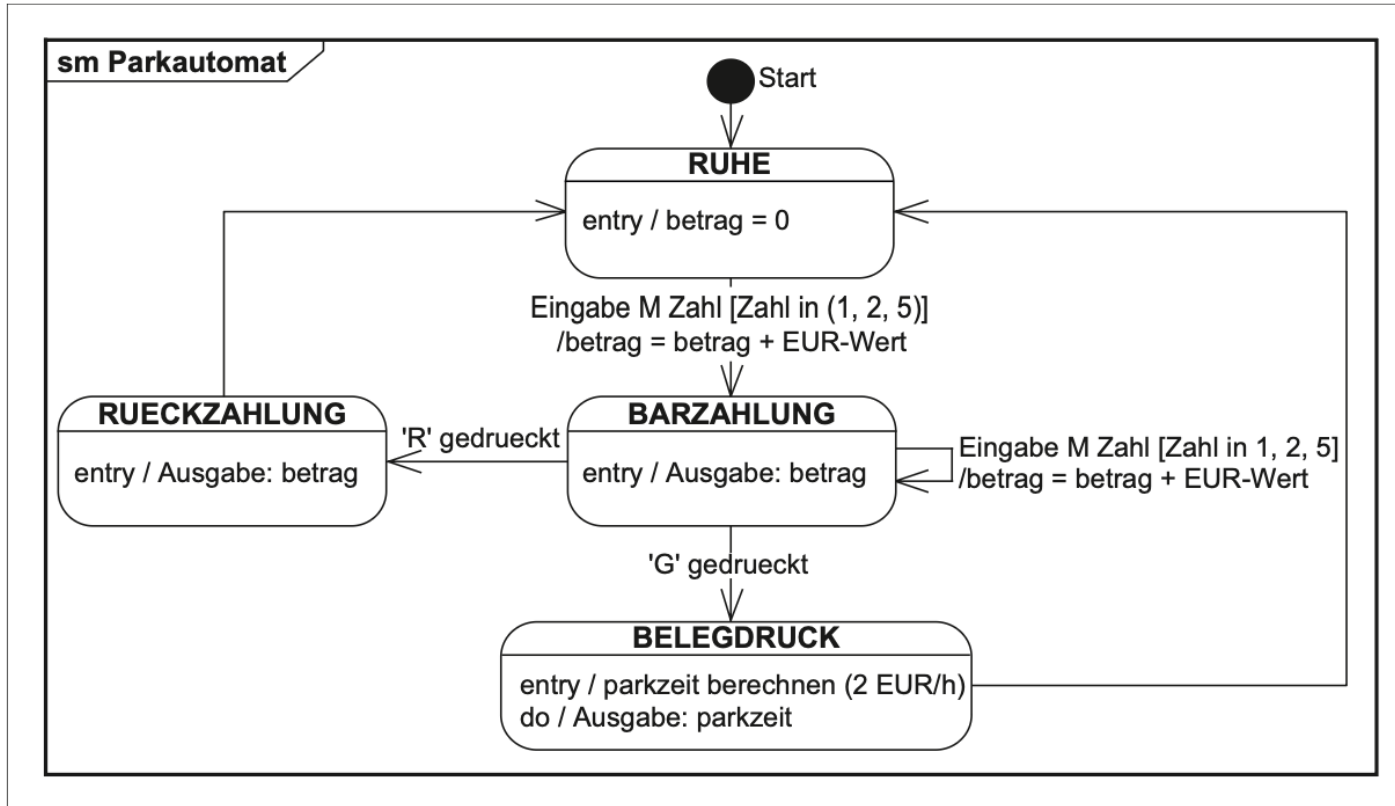
Der Zustandsautomat „Login-Prozess“ beginnt im Zustand Eingabe der Zugangsdaten. Anschließend erfolgt eine Prüfung der Zugangsdaten. Ist die Anmeldung erfolgreich, wird der Zustand Erfolgreich angemeldet erreicht. Ist die Anmeldung nicht erfolgreich, kehrt der Ablauf wieder zur Eingabe der Zugangsdaten zurück. Nach einer erfolgreichen Anmeldung endet der Zustandsautomat. Stellen Sie den Zustandsautomaten in einem Rahmen (Frame) dar und benennen Sie ihn entsprechend.

Verwenden Sie diesen Zustandsautomaten anschließend in einem übergeordneten Zustandsautomaten mit dem Namen „Online-Banking-System“. Das System befindet sich zunächst im Zustand Nicht angemeldet. Startet ein/e Kund/-in den Anmeldevorgang, wird der Zustandsautomat „Login-Prozess“ ausgeführt. Nach erfolgreichem Abschluss befindet sich das System im Zustand Angemeldet. Verlässt der/die Kund/-in das System, wechselt es wieder in den Zustand Nicht angemeldet zurück.

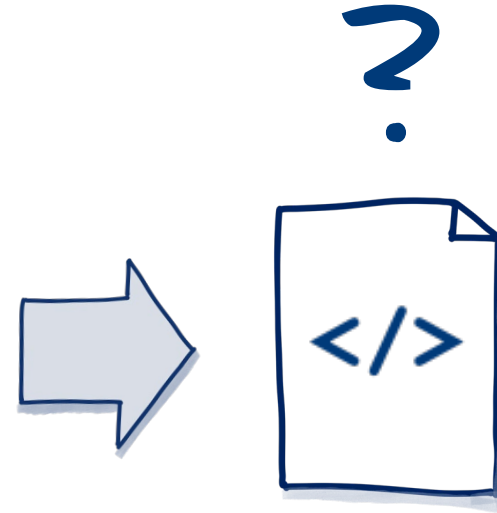
Modellieren Sie die Wiederverwendung des Zustandsautomaten mithilfe eines Unterzustandsautomatenzustands (Submachine State). Nutzen Sie außerdem geeignete Eintritts- und Austrittspunkte, um den Ablauf klar und nachvollziehbar darzustellen.

ZUSTANDSDIAGRAMME IN JAVA

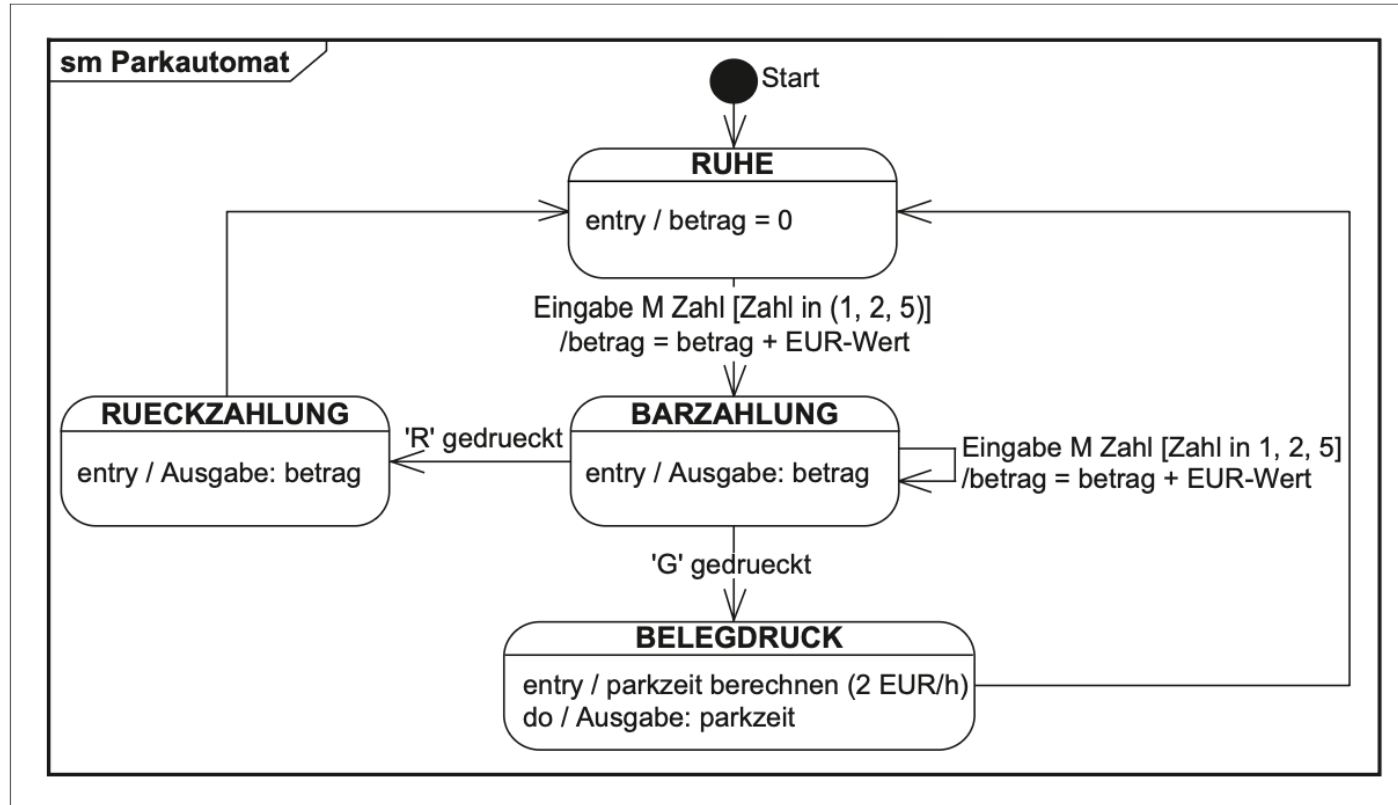
Zustandsdiagramme in Java



Bildquelle Kecher*



Das Beispiel Parkautomat

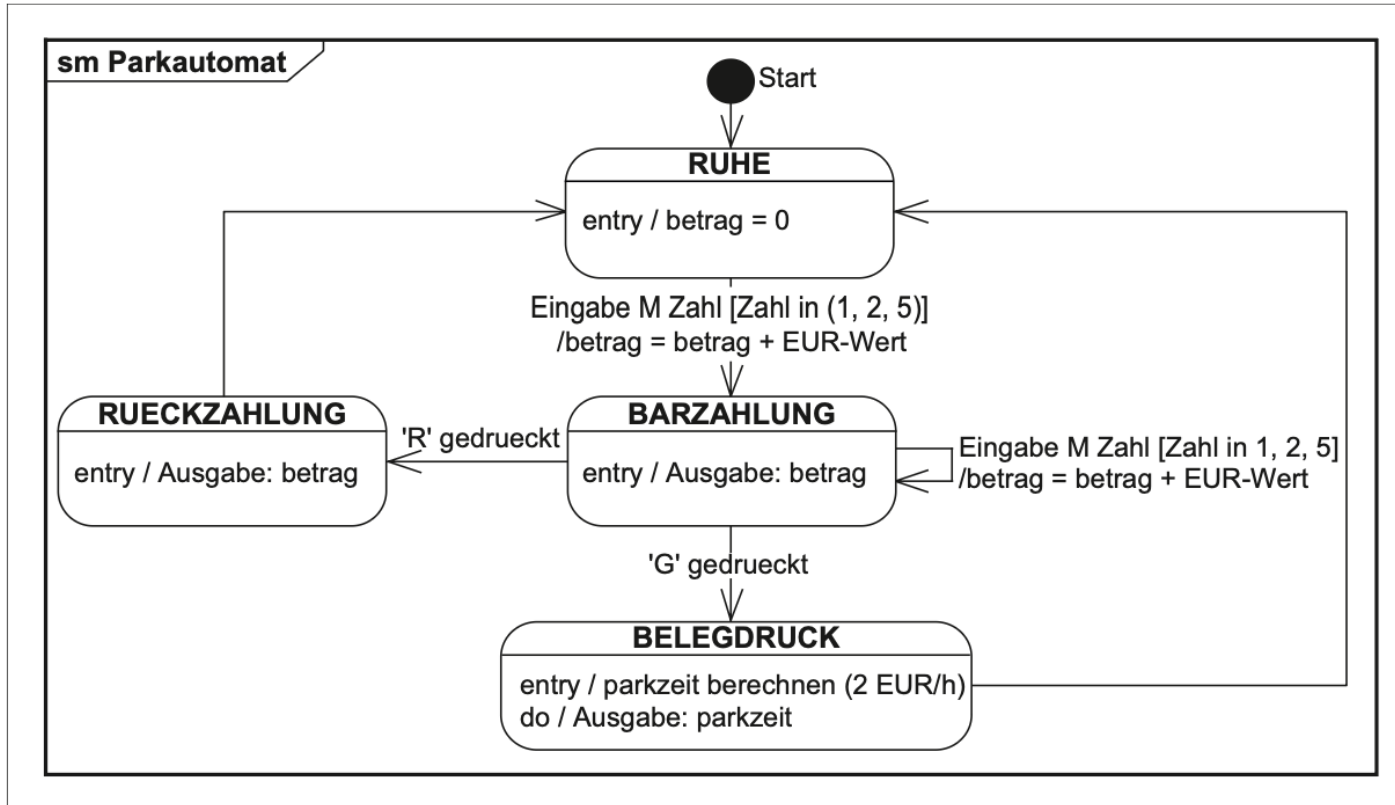


Bildquelle Kecher*

Ablauf und Zustände

- Modelliert als Zustandsautomat mit Zuständen: RUHE, BARZAHLUNG, RUECKZAHLUNG, BELEGDRUCK
- Nach Start:
 - → Zustand RUHE
 - betrag wird auf 0 gesetzt

Das Beispiel Parkautomat



Bildquelle Kecher*

Interaktion / Übergänge

- M1, M2, M5 (Münze einwerfen)
 - erhöht den Betrag
 - → Wechsel zu BARZAHLUNG
 - weitere Münzen erhöhen den Betrag weiter
- R (Abbruch / Rückgabe)
 - → Zustand RUECKZAHLUNG
 - Betrag wird ausgegeben
 - → Rückkehr zu RUHE
- G (Bestätigen / Ticket anfordern)
 - → Zustand BELEGDRUCK
 - Parkzeit wird berechnet und ausgegeben
 - → Rückkehr zu RUHE

Der Parkautomat in Java

- **Event** als eigene Klasse modelliert
- Kapselt die eingehende Nachricht (*msg*)
- Speichert optional einen zugehörigen Wert (*value*)
- Beispiel:
 - $M1 \rightarrow msg = 'M', value = 1$
 - $G \rightarrow msg = 'G', value = 0$
- Dient als einheitliche Eingabe für den Zustandsautomaten
- **Zustand**: Definiert die möglichen Zustände des Automaten

```
class Event {  
    char msg;  
    int value;  
  
    public Event(char m, int v) {  
        msg = m;  
        value = v;  
    }  
}
```

```
protected enum Zustand {  
    RUHE,  
    BARZAHLUNG,  
    RUECKZAHLUNG,  
    BELEGDRUCK  
}
```

Der Parkautomat in Java

Methode `getEvent()`

- Schnittstelle zwischen Benutzer/-in und Zustandsautomat
- Liest Eingaben ein und erzeugt ein Event-Objekt
- Ablauf:
 - Schleife läuft, bis ein gültiges Event erkannt wird
 - Eingabe wird zeichenweise ausgewertet (msg, value)
- Ergebnis:
 - Rückgabe eines validierten Events an den Automaten

```
public class Parkautomat {
    private Zustand zustand;
    private int betrag = 0;
    private Scanner scanner = new Scanner(System.in);

    protected Event getEvent() {
        while (true) {
            System.out.print("Event: ");
            String input = scanner.nextLine();

            char c0 = input.length() >= 1 ? input.charAt(0) : 0;
            char c1 = input.length() >= 2 ? input.charAt(1) : 0;

            switch (c0) {
                case 'M':
                    if (c1 == '1' || c1 == '2' || c1 == '5') {
                        return new Event(c0, c1 - '0');
                    }
                    break;

                case 'G':
                case 'R':
                    return new Event(c0, 0);
            }

            System.out.println("Ungültiges Event (M1, M2, M5, G, R)");
        }
    } ...
}
```

Der Parkautomat in Java

Zustandsoperationen

- Jeder Zustand wird durch eine eigene Operation im Code umgesetzt
- In diesen Operationen wird das Verhalten des jeweiligen Zustands implementiert

Zustandsoperation „RUHE“

- Beim Eintritt:
 - Ausführung der entry-Aktion (betrag = 0)
- Ablauf: Solange Zustand RUHE aktiv ist warten auf ein Event
- Reaktion mit Zustandswechsel

```
public class Parkautomat {  
    ...  
    private Zustand RUHE() {  
        System.out.println("\nZustand: RUHE");  
        betrag = 0;  
  
        while (zustand == Zustand.RUHE) {  
            Event e = getEvent();  
  
            if (e.msg == 'M') {  
                betrag += e.value;  
                return Zustand.BARZAHLUNG;  
            }  
        }  
        return Zustand.RUHE;  
    }  
}
```

Der Parkautomat in Java

Zustandsoperation „BARZAHLUNG“

- Eintritt: Ausgabe des aktuellen Betrags (entry-Aktion)
- Ablauf: wartet auf Event
- Reaktionen:
 - M1, M2, M5: Betrag erhöhen → bleibt in BARZAHLUNG
 - G: Wechsel zu BELEGDRUCK
 - R: Wechsel zu RUECKZAHLUNG

```
public class Parkautomat {  
    ...  
    private Zustand BARZAHLUNG() {  
        System.out.println("\nZustand: BARZAHLUNG");  
        System.out.println("Betrag: " + betrag);  
  
        while (zustand == Zustand.BARZAHLUNG) {  
            Event e = getEvent();  
  
            switch (e.msg) {  
                case 'M':  
                    betrag += e.value;  
                    return Zustand.BARZAHLUNG;  
  
                case 'G':  
                    return Zustand.BELEGDRUCK;  
  
                case 'R':  
                    return Zustand.RUECKZAHLUNG;  
            }  
        }  
        return Zustand.RUHE;  
    }  
}
```

Der Parkautomat in Java

Zustandsoperation „RUECKZAHLUNG“

- Gibt beim Eintritt den Rückzahlungsbetrag aus (entry-Aktion)
- Danach → Wechsel zurück zu RUHE

Zustandsoperation „BELEGDRUCK“

- Berechnet und gibt die Parkzeit aus
- Danach → Wechsel zurück zu RUHE

```
public class Parkautomat {  
    ...  
    private Zustand RUECKZAHLUNG() {  
        System.out.println("\nZustand: RUECKZAHLUNG");  
        System.out.println("Rückzahlung: " + betrag);  
        return Zustand.RUHE;  
    }  
  
    private Zustand BELEGDRUCK() {  
        System.out.println("\nZustand: BELEGDRUCK");  
        System.out.printf("Parkzeit: %.1f h%n", betrag / 2.0);  
        return Zustand.RUHE;  
    }  
}
```

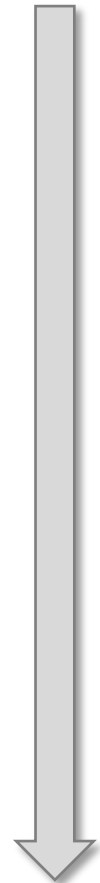

Der Parkautomat in Java

Methode start()

- Initialisierung: Start im Zustand RUHE
- Endlosschleife:
 - Automat läuft kontinuierlich (kein Endzustand)
- Ablauf:
 - abhängig vom aktuellen Zustand wird die passende Zustandsoperation aufgerufen
 - Rückgabewert = neuer Zustand
- Fehlerfall:
 - unbekannter Zustand → Fehlermeldung
 - Rücksetzung auf RUHE

```
public class Parkautomat {  
    ...  
    public void start() {  
        zustand = Zustand.RUHE;  
  
        while (true) {  
            switch (zustand) {  
                case RUHE:  
                    zustand = RUHE();  
                    break;  
  
                case BARZAHLUNG:  
                    zustand = BARZAHLUNG();  
                    break;  
  
                case BELEGDRUCK:  
                    zustand = BELEGDRUCK();  
                    break;  
  
                case RUECKZAHLUNG:  
                    zustand = RUECKZAHLUNG();  
                    break;  
            }  
        }  
    }  
}
```

Interaktion mit dem Parkautomat



Zustand: RUHE

Event: M2

Zustand: BARZAHLUNG

Betrag: 2

Event: M5

Zustand: BARZAHLUNG

Betrag: 7

Event: G

Zustand: BELEGDRUCK

Parkzeit: 3.5 h

Zustand: RUHE

Event:



ZUSAMMENFASSUNG

Grundlagen Zustandsdiagramme

- Modellieren den Lebenszyklus von Objekten

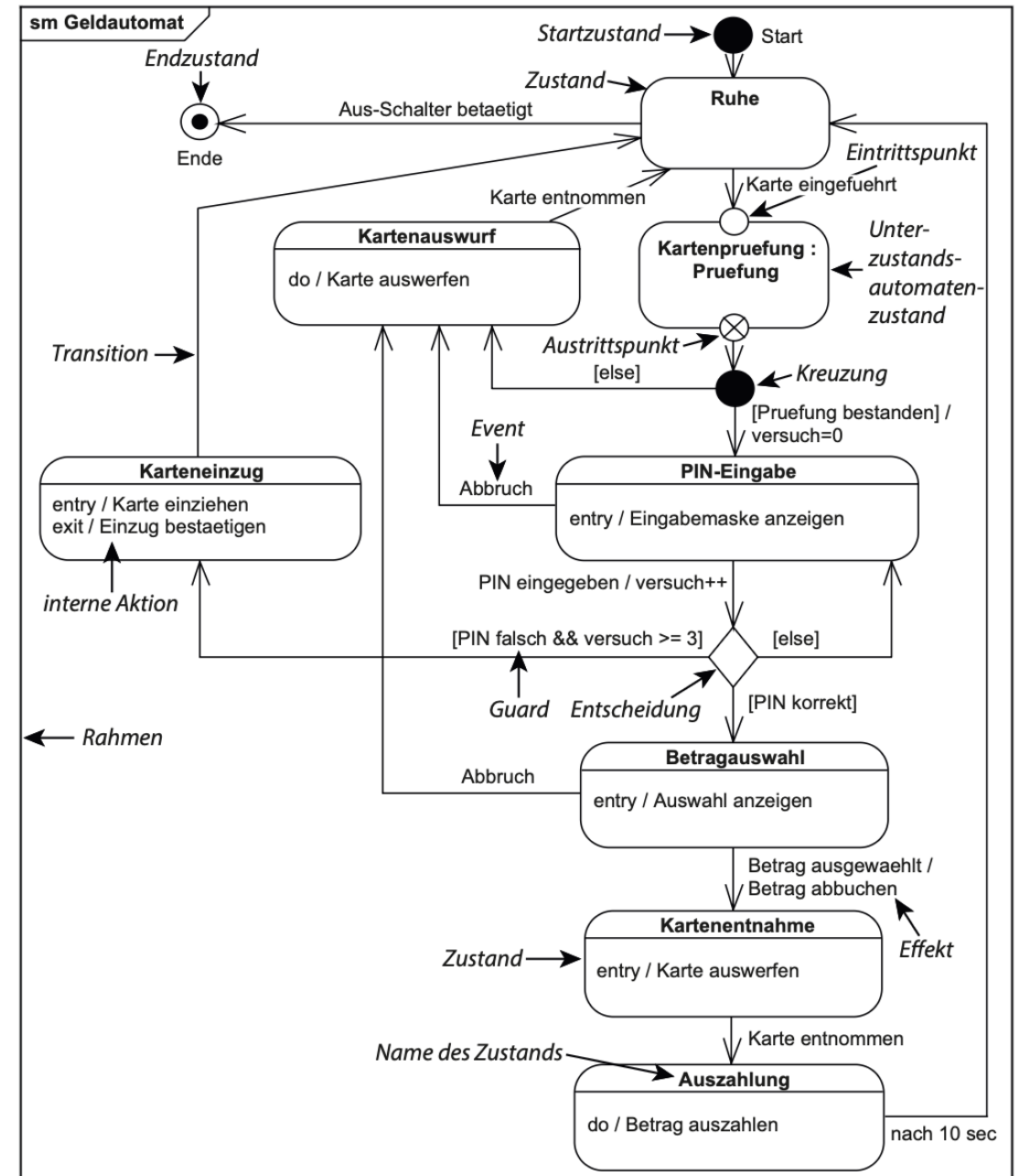
- Fokus: **Zustände** und **Zustandswechsel**

- **Zustand**

- beschreibt Situation eines Objekts
- genau ein aktiver Zustand

- **Transition**

- ausgelöst durch Event
- optional:
 - Guard [Bedingung]
 - Effekt / Aktion



Verhalten und Ereignisse

▪ Event-Typen

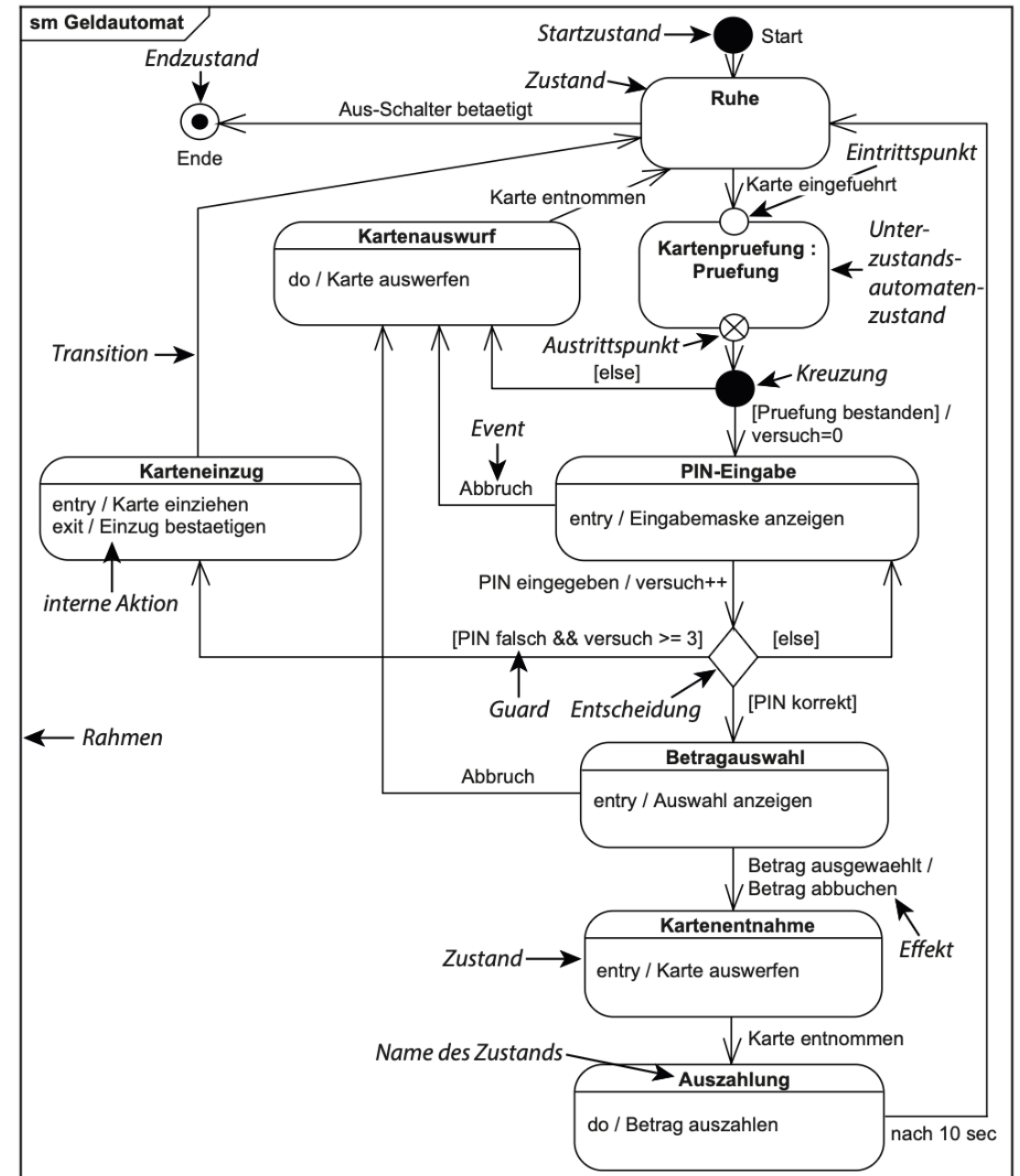
- Call, Signal, Change (when), Time (after)

▪ Reaktionen auf Events

- ignorieren
- Zustandswechsel
- interne Aktion

▪ Interne Aktionen

- entry – beim Betreten
- do – währenddessen
- exit – beim Verlassen



Struktur und Erweiterungen

▪ Verzweigungen

- Entscheidung → dynamisch
- Kreuzung → statisch

▪ Zusammengesetzte Zustände

- hierarchische Struktur
- ermöglichen Abstraktion

▪ Regionen

- Parallelität (Fork / Join)

▪ Historie

- $H / H^* = \text{„Gedächtnis“}$

▪ Wiederverwendung

- Submachine States

